

UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA
IN
METODI AVANZATI DI PROGRAMMAZIONE

Sintesi di un algoritmo per Windows PE malware detection

RELATRICE:
Prof.ssa Annalisa Appice

LAUREANDO:
Domenico Bianchini

CORRELATRICE:
Prof.ssa Giuseppina Andresini

ANNO ACCADEMICO 2025 - 2026

Indice

1 Introduzione 1
1.1 Contesto e motivazioni 1
1.2 Obiettivi della tesi 2
1.3 Struttura della tesi 2

2 File Windows PE e malware 4
2.1 Il formato Windows PE 4
2.2 Malware e tecniche di rilevamento 7
2.2.1 Tipologie di malware 7
2.2.2 Tecniche di rilevamento 8

3 Scoperta di Windows PE malware 11
3.1 Conversione di file PE in immagini 11
3.1.1 Rappresentazione in scala di grigi e RGB 11
3.1.2 Strategie di mapping 13
3.1.3 Dimensioni delle immagini 16
3.2 ResNet per classificazione di immagini 18
3.2.1 Reti neurali convoluzionali 18
3.2.2 ResNet e residual learning 19

4 Validazione empirica 22
4.1 Dataset 22
4.2 Set-up e metriche 23
4.2.1 Configurazioni sperimentali 23
4.2.2 Preparazione dei dati e addestramento 25
4.2.3 Valutazione e metriche 27

4.3 Risultati 28
4.3.1 Risultati delle configurazioni sperimentali 28
4.3.2 Confronto delle configurazioni 30
4.3.3 Discussione dei risultati 31

5 Conclusioni e sviluppi futuri 34

A Progettazione UML 36
A.1 Diagramma UML generale 36
A.2 Package main 37
A.2.1 Classe Application 37
A.3 Package pe 39
A.3.1 Classe PEFile 40
A.3.2 Enumerazione MappingType 40
A.4 Package image_representation 41
A.4.1 Classe Image 42
A.4.2 Classe astratta ImageMappingStrategy 43
A.4.3 Classe LinearMapping 44
A.4.4 Classe ZigZagMapping 45
A.4.5 Classe SerpentineMapping 45
A.5 Package dataloader 45
A.5.1 Classe ImageDataset 46
A.5.2 Classe DataLoader 47
A.6 Package model 48
A.6.1 Classe ResNetModel 48
A.7 Package utils 49
A.7.1 Classe LabelDictionary 50
A.7.2 Classe ClassificationMetrics 51

Bibliografia 53

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

La sicurezza informatica rappresenta un aspetto sempre più rilevante, considerando la crescente diffusione di sistemi e servizi digitali nei diversi ambiti della vita quotidiana. Tra le principali minacce alla sicurezza dei sistemi informatici rientrano i malware, software sviluppati con l'obiettivo di compiere azioni dannose o non autorizzate, che possono compromettere il corretto funzionamento dei sistemi e mettere a rischio i dati in essi contenuti. La diffusione dei malware e la loro continua evoluzione rendono il loro rilevamento un problema di particolare importanza nell'ambito della cybersecurity.

Esistono diversi approcci per il rilevamento dei malware, fra cui i metodi tradizionali basati sul riconoscimento di firme note e sull'analisi delle caratteristiche del software. Tuttavia, la continua comparsa di nuove varianti di malware e l'utilizzo di tecniche che ne rendono più difficile l'individuazione hanno portato allo sviluppo di approcci alternativi. In particolare, negli ultimi anni hanno assunto particolare importanza le tecniche basate su Machine Learning e Deep Learning, che permettono di individuare più efficacemente anche nuove varianti di malware, superando alcuni dei limiti dei metodi tradizionali.

Le tecniche di Deep Learning per Computer Vision possono essere usate con una rappresentazione visuale dei file, ottenuta trasformando le informazioni contenute nei file binari in immagini. In questo modo è possibile applicare tecniche di Deep Learning per la classificazione delle immagini, con l'obiettivo di distinguere file

malevoli da file benigni.

1.2 Obiettivi della tesi

L'obiettivo di questo lavoro di tesi è la sintesi di un algoritmo in grado di classificare i file Windows Portable Executable (PE) come malevoli o benigni, rappresentandoli prima sotto forma di immagini e utilizzando successivamente un modello di Deep Learning per la loro classificazione.

In particolare, questo lavoro analizza e confronta tre diverse strategie per la conversione dei file PE in immagini, ossia LINEAR, ZIGZAG e SERPENTINE, che si differenziano in base al tipo di trasformazione utilizzato per costruire l'immagine. Per ciascuna strategia vengono confrontate sia immagini RGB sia immagini in scala di grigi (*grayscale*), generate secondo due diverse configurazioni, a dimensione fissa oppure ad altezza variabile in base alla dimensione del file PE. Le immagini ottenute vengono poi utilizzate per addestrare e valutare un modello ResNet-50, con l'obiettivo di distinguere i file malevoli da quelli benigni.

Il confronto dei risultati ottenuti dalle diverse configurazioni e strategie di mapping permette infine di valutarne le prestazioni in termini di accuratezza e di individuare quale configurazione risulti più accurata nella classificazione dei file PE.

1.3 Struttura della tesi

La tesi è organizzata in cinque capitoli, il primo dedicato all'introduzione del lavoro, al suo contesto, alle motivazioni e agli obiettivi.

Il Capitolo 2 descrive il formato Windows Portable Executable (PE), analizzandone le principali caratteristiche, e introduce il concetto di malware e le principali tecniche utilizzate per il loro rilevamento.

Il Capitolo 3 descrive il processo di conversione dei file PE in immagini, presentando le diverse strategie di mapping e le configurazioni utilizzate per la loro rappresentazione, oltre a introdurre le reti neurali convoluzionali e l'architettura della ResNet utilizzata per la successiva classificazione.

Il Capitolo 4 descrive la validazione empirica della tecnica proposta, il dataset utilizzato, il set-up sperimentale e le metriche adottate per la valutazione. Nel capitolo vengono, quindi, illustrati i risultati ottenuti dalle diverse configurazioni utilizzate, con lo scopo di individuare la configurazione che permette di addestrare il modello più accurato nella classificazione dei file PE.

Infine, il Capitolo 5 riassume i principali risultati ottenuti e presenta le conclusioni del lavoro e i possibili sviluppi futuri.

Capitolo 2

File Windows PE e malware

2.1 Il formato Windows PE

Il Portable Executable (PE) è il formato utilizzato nei sistemi Windows per i file `.exe`, `.dll` e altre tipologie di file eseguibili. Inizialmente progettato come formato eseguibile standard per i sistemi Windows, il formato PE, fin dalla sua introduzione, ha subito diverse modifiche, anche in seguito al passaggio ai sistemi Windows a 64 bit. La struttura di un file PE contiene le informazioni necessarie al sistema operativo Windows per caricare e gestire il codice eseguibile. Oltre al codice da eseguire, un file PE contiene infatti metadati e altre informazioni organizzate secondo una struttura ben definita, la cui conoscenza riveste un ruolo importante nell'ambito dell'analisi dei malware. In particolare, la struttura di un file PE è organizzata in tre parti [1], come mostrato nella Figura 2.1:

- **Header**: contiene le informazioni relative al file PE e al suo caricamento in memoria. L'header è a sua volta suddiviso nelle seguenti parti:
  - **DOS Header**: rappresenta la parte iniziale del file PE e contiene informazioni relative al formato DOS. Contiene inoltre un riferimento che indica la posizione iniziale del PE Header;
  - **DOS Stub**: viene utilizzato per visualizzare un messaggio che indica se il programma può essere eseguito o meno in modalità DOS;
  - **NT Header**: contiene le principali informazioni relative al file PE ed è a sua volta suddiviso in:

    - **PE Signature**: identifica il file come Portable Executable;
    - **COFF File Header**: contiene informazioni sulle caratteristiche del file e sulle sezioni presenti;
    - **Optional Header**: contiene informazioni utilizzate durante il caricamento del file in memoria, tra cui il punto di ingresso del programma.

- **Tabella delle sezioni**: descrive le caratteristiche delle diverse sezioni presenti nel file e contiene le informazioni necessarie per mapparle in memoria e indicare se ciascuna sezione è di sola lettura oppure scrivibile;

- **Sezioni**: contengono il codice e i dati del file PE. Le sezioni comunemente presenti includono:

    - `.text`: contiene il codice eseguibile;
    - `.data`: contiene le variabili globali e statiche inizializzate;
    - `.bss`: contiene le variabili globali e statiche non inizializzate;
    - `.rsrc`: contiene le risorse utilizzate dal programma, come icone, immagini, stringhe e finestre di dialogo;
    - `.reloc`: contiene i dati necessari alla rilocazione del programma;

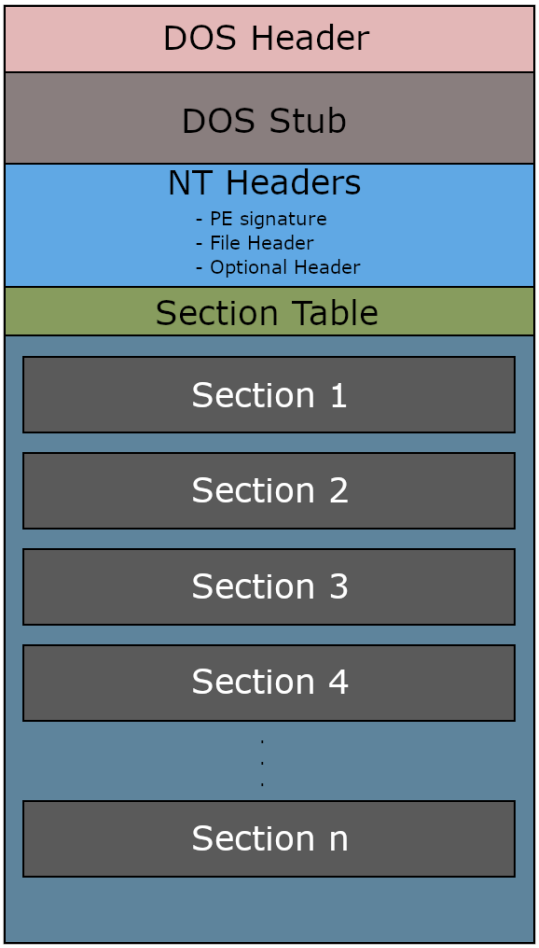


Figura 2.1: Struttura generale di un file PE, tratta da [2]

Le informazioni incluse nell'header [3], ma anche le informazioni incluse nelle sezioni sono utili nell'ambito del rilevamento dei malware. In particolare, il codice malevolo può essere nascosto all'interno di sezioni insolite oppure possono essere aggiunte ulteriori sezioni con l'obiettivo di rendere più complessa l'analisi del file. Inoltre, i file malware possono presentare anomalie nella struttura e nelle caratteristiche del formato PE rispetto ai file benigni, rendendo queste informazioni rilevanti per la loro individuazione [4].

2.2 Malware e tecniche di rilevamento

Il termine malware deriva da *malicious software* e indica un software progettato con lo scopo di compromettere la riservatezza, l'integrità o la disponibilità dei dati, delle applicazioni o del sistema operativo [5].

2.2.1 Tipologie di malware

I malware possono essere suddivisi in diverse tipologie in base alle loro caratteristiche e al loro comportamento. Inoltre, queste categorie non sono esclusive, poiché uno stesso malware può presentare contemporaneamente caratteristiche appartenenti a più tipologie [6]. Alcune categorie di malware sono descritte nel seguito:

- ***worm***: sono programmi in grado di essere eseguiti in modo indipendente e con la capacità di propagarsi verso altri sistemi. I worm possono inoltre essere utilizzati per creare *botnet*, impiegate per l'invio di messaggi di spam o per effettuare attacchi di tipo Distributed Denial of Service (DDoS), nei quali più computer infettati da worm agiscono in modo coordinato per esaurire le risorse di un sistema o la sua banda di rete.

- ***virus***: a differenza dei worm, non possono essere eseguiti in modo indipendente, ma inseriscono il proprio codice malevolo all'interno di altri programmi e si attivano quando viene eseguito il programma infettato. Possono inoltre propagarsi infettando altri file presenti nel sistema e, attraverso file presenti su server condivisi, diffondersi anche verso altri computer.

- ***Trojan horse***: sono programmi che si presentano come software legittimi, ma che eseguono operazioni malevole in background. Una volta installati, possono ad esempio scaricare altri malware, modificare le impostazioni del sistema o infettare altri file.

- ***spyware***: sono malware utilizzati per raccogliere informazioni sensibili dal sistema infetto e trasmetterle all'attaccante. Le informazioni raccolte possono comprendere credenziali di accesso, dati bancari, pagine web visitate oppure il contenuto di documenti ed e-mail.

- ***bot***: permettono a un attaccante, chiamato *bot master*, di controllare da remoto il sistema infetto. Un insieme di bot controllati dallo stesso *bot master* costituisce una *botnet*, che può essere utilizzata, ad esempio, per l'invio di messaggi di spam.

- ***rootkit***: sono malware la cui principale caratteristica è quella di nascondere determinate informazioni all'utente del sistema, come processi, file o connessioni di rete. Per questo motivo, molti malware utilizzano tecniche proprie dei rootkit per nascondere la propria presenza all'interno del sistema.

2.2.2 Tecniche di rilevamento

Il rilevamento dei malware può essere effettuato attraverso diverse tecniche. Uno degli approcci tradizionalmente utilizzati, anche dagli antivirus, è quello basato sulle firme, in cui i file vengono confrontati con un insieme di firme associate a malware già conosciuti. Questo approccio presenta tuttavia alcuni limiti, in particolare le firme vengono generalmente create da analisti, richiedendo tempo e potendo essere soggette a errori. Inoltre, i metodi basati sulle firme non sono in grado di rilevare minacce sconosciute per le quali non è ancora disponibile una firma, dato che solo quando viene individuata una nuova minaccia questa può essere analizzata e può essere creata una nuova firma, che viene successivamente aggiunta al database utilizzato per il rilevamento [6].

Per affrontare alcuni dei limiti dei metodi basati sulle firme, sono state sviluppate altre tecniche per il rilevamento dei malware. In particolare, è possibile distinguere due principali approcci: l'analisi dinamica e l'analisi statica, in base all'esecuzione o meno del programma durante la raccolta delle feature, ovvero delle caratteristiche utilizzate per effettuare l'analisi [3]. In particolare, l'analisi dinamica prevede l'esecuzione del programma e l'osservazione del suo comportamento durante l'esecuzione. In questo modo possono essere raccolte diverse informazioni, tra cui le sequenze delle chiamate API, le comunicazioni di rete, lo stato della memoria e della CPU. Questo tipo di analisi può inoltre contribuire al rilevamento di malware precedentemente sconosciuti e risulta meno vulnerabile alle tecniche di offuscamento del codice. Tuttavia, l'analisi dinamica presenta alcuni limiti, dato che

ogni campione deve essere eseguito per un determinato periodo di tempo e l'analisi può riguardare soltanto un singolo percorso di esecuzione del programma [4].

A differenza dell'analisi dinamica, l'analisi statica permette di analizzare un programma senza eseguirlo e, per questo motivo, richiede tempi inferiori per l'analisi. Dato che generalmente il codice sorgente dei malware non è disponibile, le tecniche di analisi statica applicabili devono ricavare le informazioni direttamente dalla rappresentazione binaria del programma. Anche l'analisi statica presenta alcuni limiti, dato che l'esame della rappresentazione binaria di un programma può portare ad ignorare informazioni rese note durante l'analisi comportamentale. Inoltre, i malware possono utilizzare tecniche in grado di modificare il programma e ostacolare questo tipo di analisi [6].

La Tabella 2.1 mostra un confronto tra le principali caratteristiche dell'analisi statica e dell'analisi dinamica.

Tabella 2.1: Confronto tra analisi statica e dinamica dei malware, rielaborato da [7]

Caratteristica	**Analisi statica**	**Analisi dinamica**
Esecuzione	Non richiesta	Richiesta
Informazioni analizzate	Rappresentazione binaria del programma	Comportamento durante l'esecuzione
Vantaggi	Tempi di analisi inferiori	Può contribuire al rilevamento di malware sconosciuti ed è meno vulnerabile alle tecniche di offuscamento
Limiti	Può essere ostacolata da tecniche di packing o cifratura	Può richiedere tempi di analisi maggiori
Rischi	Non comporta i rischi legati all'esecuzione del malware	Richiede un ambiente isolato per limitare i rischi legati all'esecuzione del malware

I limiti degli approcci descritti in precedenza, insieme alla crescente complessità e quantità dei malware, hanno favorito l'utilizzo di tecniche basate sul Machine Learning e, più recentemente, sul Deep Learning. In particolare, queste tecniche

permettono di apprendere modelli e pattern a partire da una rappresentazione dei file Windows PE (malware e goodware), con l'obiettivo di supportarne il rilevamento e l'analisi [8].

Capitolo 3

Scoperta di Windows PE malware

3.1 Conversione di file PE in immagini

Un possibile approccio per l'analisi statica dei malware consiste nel rappresentare il contenuto binario dei file sotto forma di immagini. In questo modo, la sequenza di byte che compone un file può essere convertita in una rappresentazione da elaborare con tecniche di computer vision per la classificazione di immagini.

3.1.1 Rappresentazione in scala di grigi e RGB

Nataraj et al. [9] hanno proposto un approccio in cui il contenuto binario di un file viene riorganizzato in una matrice e rappresentato sotto forma di un'immagine in scala di grigi. Gli autori hanno osservato che malware appartenenti alla stessa famiglia possono presentare immagini simili a livello di layout e texture. A partire da questa osservazione, hanno quindi utilizzato caratteristiche estratte dalle immagini per effettuare la classificazione, senza richiedere il disassemblaggio del codice o l'esecuzione dei file analizzati. In particolare, la conversione in immagine del file PE viene eseguita leggendo il contenuto del file come una sequenza di byte, ciascuno dei quali viene rappresentato attraverso un valore intero compreso tra 0 e 255. Nel caso di un'immagine in scala di grigi, ogni byte viene associato a un pixel, il cui valore determina il livello di intensità, dove 0 corrisponde al nero e 255 al bianco. I valori ottenuti vengono quindi organizzati in una matrice bidimensionale che costituisce l'immagine. Questo processo di trasformazione è schematizzato nella Figura 3.1.

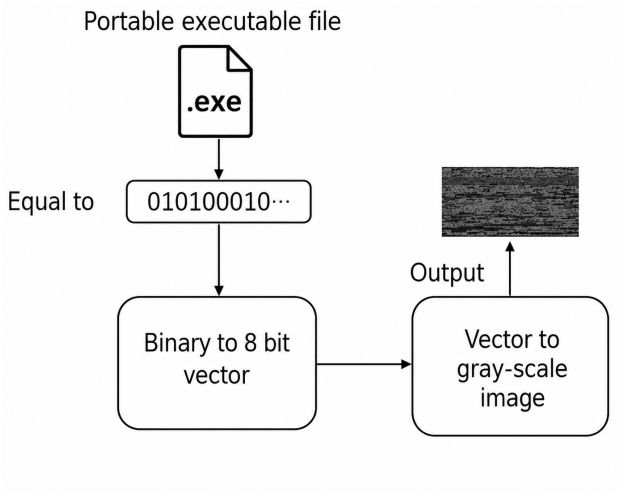
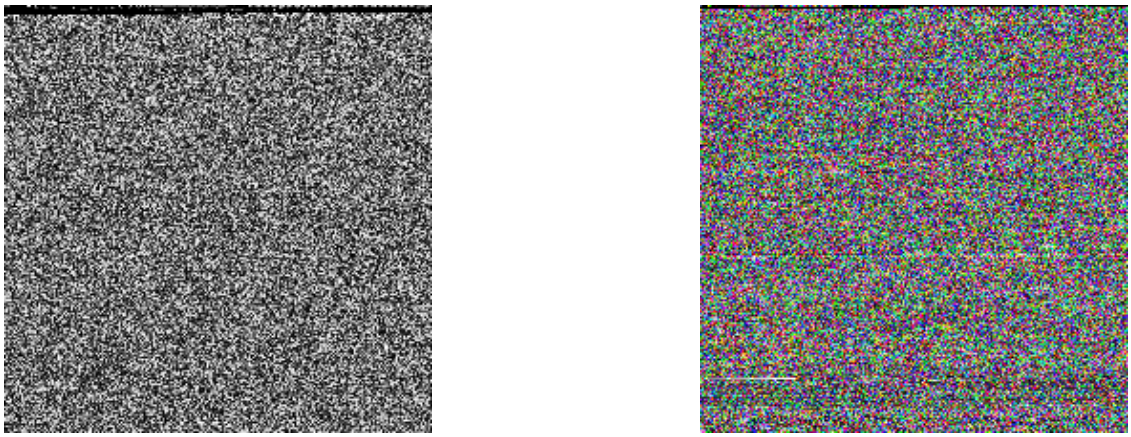


Figura 3.1: Fasi della conversione di un file PE in un'immagine in scala di grigi, tratta da [3]

In alternativa alla rappresentazione in scala di grigi, i byte del file PE possono essere utilizzati per costruire un'immagine RGB. In questo caso, ciascun pixel è composto da tre canali, rosso, verde e blu, ognuno dei quali assume, come nel caso precedente, un valore compreso tra 0 e 255. Guo et al. [10] utilizzano una rappresentazione RGB in cui tre byte consecutivi vengono raggruppati e associati, nello stesso ordine, ai tre canali di un pixel. Il primo byte determina quindi il valore del canale rosso, il secondo quello del canale verde e il terzo quello del canale blu. Le due rappresentazioni sono mostrate nella Figura 3.2.



(a) Rappresentazione in scala di grigi (b) Rappresentazione RGB

Figura 3.2: Rappresentazioni in scala di grigi e RGB dello stesso file PE

Nel presente lavoro di tesi vengono considerate entrambe le rappresentazioni di file PE come immagini, in scala di grigi e RGB. Una volta ottenuti i valori dei pixel a partire dai byte del file PE, occorre stabilire come disporli all'interno della matrice che costituisce l'immagine.

3.1.2 Strategie di mapping

Linh et al. [11] analizzano differenti modalità di disposizione dei pixel, considerando tre diverse strategie di mapping denominate Linear, Zigzag e Serpentine.

Il mapping **LINEAR** dispone i pixel seguendo l'ordine originale dei byte del file PE, procedendo da sinistra verso destra e dall'alto verso il basso. Questa disposizione, tuttavia, interrompe la continuità tra righe consecutive, poiché l'ultimo pixel di una riga e il primo della successiva, pur corrispondendo a byte consecutivi, non risultano adiacenti nell'immagine. Il funzionamento del mapping LINEAR è descritto nell'Algoritmo 3.1.

Algoritmo 3.1 Pseudocodice del mapping LINEAR

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for y = 0 to height − 1 do
4:     for x = 0 to width − 1 do
5:         for c = 0 to channels − 1 do
6:             if index < length(byteData) then
7:                 image[y][x][c] = byteData[index]
8:                 index = index + 1
```

Il mapping **ZIGZAG** dispone invece i pixel alternando la direzione di percorrenza delle righe. Una riga viene quindi percorsa da sinistra verso destra, mentre quella successiva da destra verso sinistra. In questo modo viene mantenuta la continuità tra righe consecutive, poiché i pixel corrispondenti a byte consecutivi rimangono adiacenti nell'immagine. Il funzionamento del mapping ZIGZAG è descritto nell'Algoritmo 3.2.

Algoritmo 3.2 Pseudocodice del mapping ZIGZAG

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for y = 0 to height − 1 do
4:     if y mod 2 = 0 then
5:         xRange = 0 to width − 1
6:     else
7:         xRange = width − 1 to 0
8:     for x in xRange do
9:         for c = 0 to channels − 1 do
10:            if index < length(byteData) then
11:                image[y][x][c] = byteData[index]
12:                index = index + 1
```

Il mapping **SERPENTINE** utilizza infine un percorso basato sulle diagonali della matrice, alternandone la direzione di percorrenza. Questa disposizione consente di mantenere adiacenti nell'immagine i pixel corrispondenti a byte consecutivi, distribuendoli però lungo le diagonali anziché lungo le righe. Il funzionamento del mapping SERPENTINE è descritto nell'Algoritmo 3.3.

Algoritmo 3.3 Pseudocodice del mapping SERPENTINE

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for diagonal = 0 to height + width − 2 do
4:     startY = max(0, diagonal − (width − 1))
5:     endY = min(height − 1, diagonal)
6:     if diagonal mod 2 = 0 then
7:         yRange = startY to endY
8:     else
9:         yRange = endY to startY
10:    for y in yRange do
11:        x = diagonal − y
12:        for c = 0 to channels − 1 do
13:            if index < length(byteData) then
14:                image[y][x][c] = byteData[index]
15:                index = index + 1
```

Nel caso dei file PE, questa disposizione permette inoltre di concentrare in una stessa area dell'immagine le informazioni appartenenti alla parte iniziale del file,

invece di distribuirle lungo le prime righe dell'immagine come avviene nei mapping precedenti. La disposizione dei pixel nelle tre strategie di dislocazione dei pixel nella trasformazione di file PE in immagini è illustrata nelle Figure 3.3 e 3.4, mentre la Tabella 3.1 mette a confronto le caratteristiche delle diverse strategie.

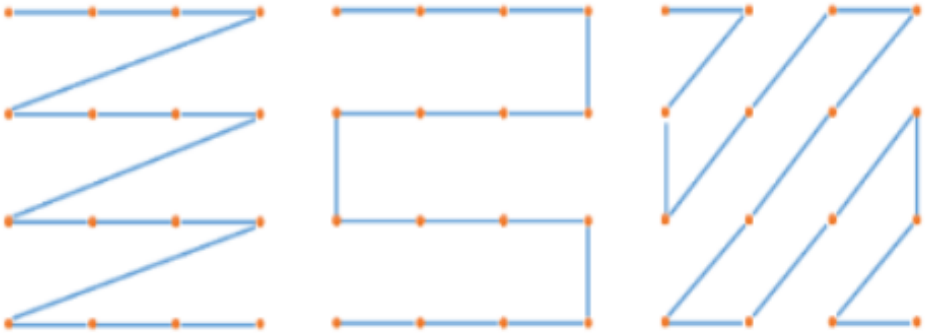


Figura 3.3: Disposizione dei pixel secondo i mapping Linear, Zigzag e Serpentine, tratta da [11]



Figura 3.4: Esempi di immagini malware ottenute utilizzando rispettivamente i mapping Linear, Zigzag e Serpentine, tratte da [11]

Tabella 3.1: Confronto tra le strategie di mapping considerate

Mapping	Disposizione	Caratteristiche
LINEAR	Da sinistra verso destra, riga per riga	Mantiene l'ordine sequenziale dei byte, ma interrompe la continuità tra righe consecutive
ZIGZAG	Direzione alternata tra righe consecutive	Mantiene la continuità tra righe consecutive, preservando l'adiacenza tra i pixel corrispondenti a byte consecutivi
SERPENTINE	Percorso alternato lungo le diagonali	Mantiene l'adiacenza tra byte consecutivi e concentra la parte iniziale del file in una stessa area dell'immagine

3.1.3 Dimensioni delle immagini

Un ulteriore aspetto da considerare nella conversione riguarda la scelta delle dimensioni dell'immagine, dato che i file PE possono avere dimensioni differenti e, di conseguenza, anche il numero di pixel ottenuti a partire dalla sequenza di byte può variare da un file all'altro.

Un possibile approccio consiste nel fissare a priori le dimensioni dell'immagine, stabilendo quindi un numero massimo di pixel disponibili. Se i dati ottenuti dal file non sono sufficienti a riempire l'intera immagine, è possibile applicare lo zero-padding, completando i pixel rimanenti con valori pari a zero. Al contrario, se la quantità di dati supera la capacità dell'immagine, i byte in coda al file che non possono essere rappresentati vengono eliminati mediante troncamento [12].

Un approccio alternativo consiste nell'adattare le dimensioni dell'immagine alla quantità di dati da rappresentare, in modo da preservare l'intera sequenza di byte ed evitare il troncamento. Nataraj et al. [9], in particolare, scelgono la larghezza dell'immagine in base alla dimensione del file, utilizzando diversi intervalli prestabiliti, mentre l'altezza dipende dal rapporto tra il numero di byte da rappresentare e la larghezza scelta. Gli intervalli di dimensione dei file e le corrispondenti larghezze utilizzate sono riportati nella Tabella 3.2.

Tabella 3.2: Larghezze delle immagini utilizzate in funzione della dimensione dei file, tratta da [9]

Dimensione file	**Larghezza immagine**
$<$ 10 kB	32
10 kB – 30 kB	64
30 kB – 60 kB	128
60 kB – 100 kB	256
100 kB – 200 kB	384
200 kB – 500 kB	512
500 kB – 1000 kB	768
$>$ 1000 kB	1024

In questa tesi si considerano entrambe le strategie per la definizione delle dimensioni delle immagini, ossia, immagini a dimensione fissa oppure immagini ad altezza variabile. Nella prima strategia le immagini hanno dimensioni fisse pari a 224×224 pixel. Nella seconda, invece, a differenza dell'approccio adottato da Nataraj et al. [9], in cui la larghezza viene scelta in funzione della dimensione del singolo file, la larghezza viene determinata una sola volta a partire dalla lunghezza media dei file del dataset e dal numero di canali della rappresentazione e viene quindi mantenuta fissa per tutti i file appartenenti alla stessa configurazione. La larghezza è stata calcolata tramite la seguente formula:

$$\text{larghezza} = \left\lceil \sqrt{\frac{\text{lunghezza media dei file}}{\text{numero di canali}}} \right\rceil .$$

Dividendo la lunghezza media dei file per il numero di canali si ottiene il numero medio di pixel necessari, considerando che a ciascun pixel corrisponde un byte nella rappresentazione in scala di grigi e tre byte nella rappresentazione RGB. La radice quadrata di tale valore viene quindi utilizzata per determinare la larghezza dell'immagine. Una volta fissata la larghezza, per ciascun file viene prima calcolato il numero di pixel necessari in base al numero di byte del file e al numero di canali:

$$\text{numero di pixel} = \left\lceil \frac{\text{numero di byte del file}}{\text{numero di canali}} \right\rceil .$$

Nel caso di immagini RGB, se il numero di byte del file non è multiplo di tre, l'arrotondamento per eccesso permette di considerare anche gli eventuali byte rimanenti. L'altezza viene quindi determinata individualmente per ciascun file dividendo il numero di pixel necessari per la larghezza e arrotondando il risultato per eccesso:

$$\text{altezza} = \left\lceil \frac{\text{numero di pixel necessari}}{\text{larghezza}} \right\rceil .$$

3.2 ResNet per classificazione di immagini

Una volta rappresentati i file PE sotto forma di immagini, il problema della distinzione tra file benigni e malevoli può essere affrontato come un problema di classificazione di immagini. A questo scopo possono essere utilizzate le reti neurali convoluzionali (Convolutional Neural Networks, CNN), modelli particolarmente adatti all'elaborazione di dati organizzati sotto forma di immagini.

3.2.1 Reti neurali convoluzionali

Le reti CNN [13] sono costituite da una successione di livelli che elaborano progressivamente i dati ricevuti in input. Un ruolo centrale è svolto nelle reti CNN dai livelli convoluzionali, nei quali dei filtri analizzano porzioni dell'immagine, andando a produrre delle feature map che evidenziano determinate caratteristiche presenti nell'immagine. Lo stesso filtro viene utilizzato nelle diverse parti dell'immagine, permettendo di individuare una determinata caratteristica indipendentemente dalla sua posizione. Attraverso la successione di più livelli, la rete è in grado di individuare caratteristiche progressivamente più complesse dell'immagine. I primi livelli possono individuare caratteristiche semplici, come colori e contorni, mentre i livelli successivi combinano tali informazioni per riconoscere forme e strutture più complesse. Il risultato prodotto dai livelli convoluzionali viene generalmente elaborato mediante l'utilizzo di una funzione di attivazione non lineare, come la Rectified Linear Unit (ReLU). Questa funzione assegna un valore pari a zero agli input negativi e mantiene invariati quelli positivi, introducendo non linearità nella rete e permettendo di apprendere pattern complessi e velocizzare l'addestramento. Ai livelli convoluzionali possono inoltre essere alternati livelli di pooling, utilizzati per ridurre la dimensionalità delle feature map. L'operazione di pooling considera delle

regioni dell'input e ne aggrega i valori, producendo una rappresentazione di dimensioni inferiori e permettendo quindi di ridurre la complessità computazionale della rete. Tra le principali operazioni di pooling vi sono il max pooling, che seleziona il valore massimo presente in ciascuna regione considerata, e l'average pooling, che ne calcola invece il valore medio [13]. Nella parte finale di una CNN possono essere presenti uno o più livelli completamente connessi (*fully connected*), nei quali ogni neurone è collegato a tutti i neuroni del livello precedente. Questi livelli utilizzano le caratteristiche estratte dai livelli precedenti per produrre l'output utilizzato per la classificazione. Nella Figura 3.5 viene mostrata la tipica architettura di una rete neurale convoluzionale e la successione dei livelli descritti.

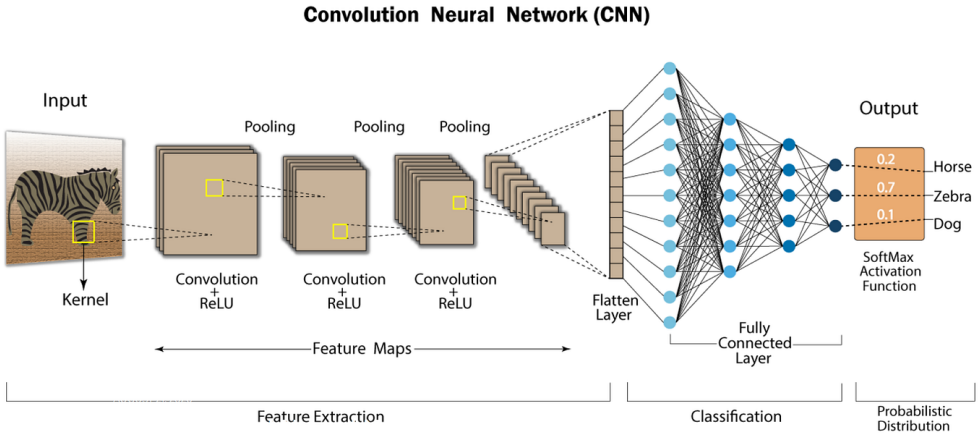


Figura 3.5: Esempio della struttura di una rete neurale convoluzionale, tratta da [14]

3.2.2 ResNet e residual learning

Nel corso degli anni sono state sviluppate diverse architetture di reti neurali convoluzionali, caratterizzate da strutture e profondità differenti. Tra queste rientra la Residual Network (ResNet), proposta da He et al. [15], che consente l'addestramento di reti estremamente profonde, cioè caratterizzate da un elevato numero di livelli. L'aumento della profondità di una rete neurale non comporta tuttavia necessariamente un miglioramento delle prestazioni. Con l'aumentare del numero di livelli può infatti verificarsi il cosiddetto *degradation problem*, per cui l'accuratezza della rete smette inizialmente di migliorare e successivamente tende a diminuire. Come osservato dagli autori, questo fenomeno non è necessariamente dovuto all'overfitting, poiché l'aggiunta di ulteriori livelli può portare anche a un peggioramento

delle prestazioni durante l'addestramento. Per affrontare questo problema, una rete ResNet usa il concetto di *residual learning*. In una rete tradizionale, una successione di livelli apprende direttamente una mappatura $H(x)$ a partire dall'input x. ResNet modifica questo approccio facendo apprendere ai livelli una funzione residua $F(x)$, che rappresenta la differenza tra la mappatura desiderata e l'input:

$$F(x) = H(x) - x.$$

La mappatura originale può quindi essere espressa come formulato nella seguente formula:

$$H(x) = F(x) + x.$$

In questo modo, invece di apprendere direttamente la mappatura $H(x)$, la successione di livelli apprende la funzione residua $F(x)$ rispetto all'input. Questa formulazione viene realizzata attraverso le *shortcut connections*, collegamenti che saltano uno o più livelli della rete. Nel caso più semplice, la shortcut lascia invariato l'input x e lo aggiunge direttamente all'output della funzione residua $F(x)$. Queste connessioni non introducono parametri aggiuntivi né aumentano la complessità computazionale della rete. Nella Figura 3.6 è mostrata la struttura tipica di un blocco residuo con questa configurazione.

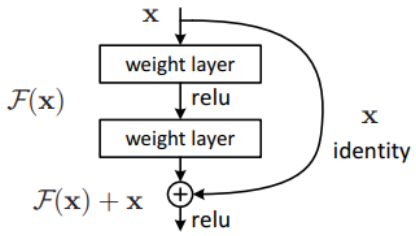


Figura 3.6: Schema di un blocco residuo con shortcut connection, tratto da [15]

Esistono diverse architetture basate sul residual learning, in questa tesi è stata utilizzata ResNet-50, che è una rete ResNet composta da 50 livelli. Questa architettura utilizza un design a collo di bottiglia (*bottleneck design*), nel quale ciascun blocco residuo è composto da tre livelli convoluzionali con filtri rispettivamente di dimensione 1×1, 3×3 e 1×1 [15]. I livelli convoluzionali con filtri 1×1, posti

rispettivamente prima e dopo quello con filtri 3×3, permettono di ridurre e successivamente ripristinare la dimensionalità dei canali, riducendo così la complessità computazionale del blocco, come illustrato nella Figura 3.7.

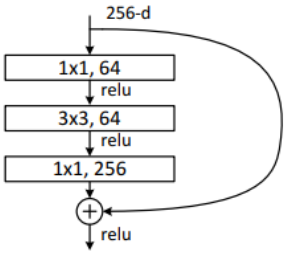


Figura 3.7: Schema del blocco bottleneck utilizzato in ResNet-50, tratto da [15]

La struttura complessiva di ResNet-50 è organizzata in più gruppi di blocchi residuali. Dopo i livelli iniziali di convoluzione e pooling, la rete presenta quattro gruppi di blocchi bottleneck, ripetuti rispettivamente 3, 4, 6 e 3 volte. La parte finale della rete comprende un livello di average pooling seguito da un livello fully connected utilizzato per la classificazione. Nella Figura 3.8 viene illustrato il confronto tra la struttura a 50 livelli e le altre configurazioni proposte in [15].

layer name	output size	18-layer	34-layer	50-layer	101-layer
conv1	112×112	7×7, 64, stride 2			
conv2_x	56×56	3×3 max pool, stride 2			
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix} \times 23$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax			

Figura 3.8: Confronto dell'architettura ResNet-50 con altre architetture ResNet, tratto da [15]

Capitolo 4

Validazione empirica

4.1 Dataset

Per la validazione sperimentale è stato utilizzato il *PE Malware Machine Learning Dataset* di Practical Security Analytics [16]. Il dataset comprende, oltre ai file PE, un file CSV contenente diverse informazioni associate a ciascun file, tra cui la lunghezza in byte e l'etichetta che indica l'appartenenza alla *Whitelist* o alla *Blacklist*, permettendo di distinguere rispettivamente i file benigni da quelli malevoli. Queste caratteristiche rendono il dataset adatto per la validazione sperimentale, poiché mette a disposizione sia i file PE originali, necessari per la loro conversione in immagini, sia le relative etichette utilizzate per la classificazione. Il dataset è composto da 201.549 file PE, di cui 86.812 appartenenti alla *Whitelist* e quindi considerati benigni, e 114.737 appartenenti alla *Blacklist* e quindi considerati malevoli. Ai fini della validazione sperimentale, il dataset è stato suddiviso in un training set, contenente l'80% dei campioni, e un test set, contenente il restante 20%. La suddivisione è stata effettuata in modo stratificato rispetto alla classe, così da mantenere la proporzione tra file benigni e malevoli presente nel dataset originale, come mostrato nella Tabella 4.1.

Tabella 4.1: Distribuzione dei campioni nei set di training e test

Set	Benigni	Malevoli	Totale
Training	69.450	91.789	161.239
Test	17.362	22.948	40.310
Totale	86.812	114.737	201.549

Il training set è stato utilizzato per l'addestramento dei modelli, mentre il test set è stato utilizzato per valutare le prestazioni dei modelli addestrati in termini di accuratezza.

4.2 Set-up e metriche

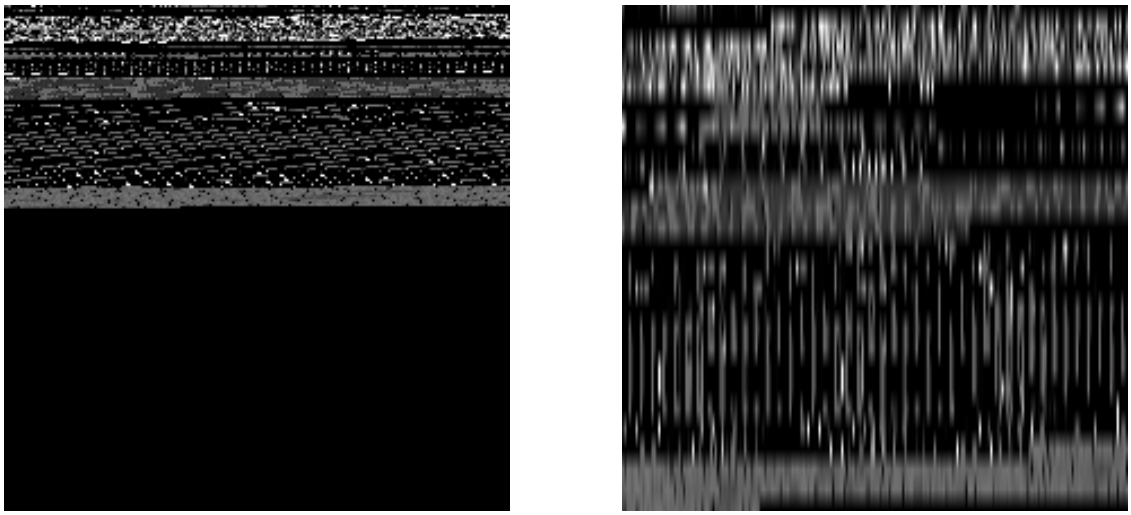
4.2.1 Configurazioni sperimentali

A partire dai file PE presenti nel training set e nel test set sono state generate le immagini utilizzate per la validazione sperimentale. Sono state considerate tutte le combinazioni tra le tre strategie di mapping, **LINEAR**, **ZIGZAG** e **SERPENTINE**, le rappresentazioni RGB e in scala di grigi e le due strategie relative alle dimensioni delle immagini. Nella prima configurazione le immagini hanno dimensione fissa pari a 224×224 pixel, mentre nella seconda hanno larghezza fissa e altezza variabile in funzione della dimensione del file PE. Sono state quindi valutate complessivamente 12 configurazioni sperimentali, come riportato nella Tabella 4.2.

Tabella 4.2: Configurazioni considerate nella validazione sperimentale

Mapping	**Rappresentazione**	**Larghezza**	**Altezza**
LINEAR	RGB	224	224
ZIGZAG	RGB	224	224
SERPENTINE	RGB	224	224
LINEAR	GRAY	224	224
ZIGZAG	GRAY	224	224
SERPENTINE	GRAY	224	224
LINEAR	RGB	457	Variabile
ZIGZAG	RGB	457	Variabile
SERPENTINE	RGB	457	Variabile
LINEAR	GRAY	792	Variabile
ZIGZAG	GRAY	792	Variabile
SERPENTINE	GRAY	792	Variabile

Nel caso delle immagini a dimensione fissa, i file PE sono stati convertiti direttamente in immagini di 224×224 pixel, che è la dimensione utilizzata in input dal modello ResNet-50 adottato negli esperimenti e quindi non è stato necessario effettuare un successivo ridimensionamento. Per le immagini ad altezza variabile, invece, la larghezza è stata determinata a partire dalla lunghezza media dei 201.549 file PE del dataset, che è risultata pari a 626.445,12 byte. Da questo valore sono state ottenute larghezze pari a 457 pixel per la rappresentazione RGB e a 792 pixel per quella in scala di grigi, secondo il procedimento descritto nel Capitolo 3, mentre l'altezza varia in base alla dimensione del singolo file PE. Le immagini ad altezza variabile sono state successivamente ridimensionate a 224×224 pixel prima di essere fornite al modello. Il ridimensionamento è stato effettuato mediante interpolazione bilineare, utilizzando `nn.Upsample` di PyTorch [17]. La Figura 4.1 mostra il risultato della conversione dello stesso file PE con le strategie a dimensione fissa e ad altezza variabile.



(a) Dimensione fissa 224×224

(b) Altezza variabile dopo il resize a 224×224

Figura 4.1: Immagini ottenute dallo stesso file PE con mapping LINEAR in scala di grigi nelle strategie a dimensione fissa e ad altezza variabile

Come si può osservare dalla Figura 4.1, nella configurazione a dimensione fissa è visibile la parte dell'immagine generata mediante zero-padding, mentre nella configurazione ad altezza variabile l'immagine appare maggiormente allungata in altezza in seguito al resize a 224×224 pixel.

4.2.2 Preparazione dei dati e addestramento

Durante la conversione dei file PE è stato inoltre generato, per il training set e per il test set, un file CSV contenente il percorso di ciascuna immagine e la relativa etichetta. In particolare, l'etichetta 0 identifica i file benigni, mentre l'etichetta 1 identifica i file malevoli. Questi file CSV sono stati successivamente utilizzati per associare ogni immagine alla classe corrispondente durante il caricamento dei dati. Per il caricamento e la preparazione delle immagini sono stati utilizzati `Dataset` e `DataLoader` di PyTorch [17]. Il dataset associa ciascuna immagine alla relativa etichetta utilizzando le informazioni contenute nei file CSV, mentre il DataLoader organizza i campioni nei batch da fornire al modello durante il training e il test. Durante il caricamento, tutte le immagini sono state lette in formato RGB utilizzando la funzione `read_image` di Torchvision [18]. Nel caso delle immagini originariamente in scala di grigi, i valori dei pixel sono stati quindi replicati sui

tre canali, in modo da ottenere un'immagine compatibile con i tre canali di input utilizzati da ResNet-50. Prima di essere fornite al modello, i valori dei pixel delle immagini sono stati convertiti nell'intervallo [0,1] e successivamente normalizzati utilizzando, per i tre canali, i valori medi $[0.485, 0.456, 0.406]$ e le deviazioni standard $[0.229, 0.224, 0.225]$, utilizzati dal modello `microsoft/resnet-50` [19]. Il training set descritto nella Sezione 4.1 è stato ulteriormente suddiviso, utilizzando l'80% dei campioni per l'addestramento e il restante 20% per la validazione. Il validation set così ottenuto è stato utilizzato per valutare le prestazioni del modello durante le diverse epoche dell'addestramento.

Per tutte le configurazioni sperimentali è stato utilizzato, tramite la libreria Hugging Face Transformers, il modello `microsoft/resnet-50`, pre-addestrato sul dataset ImageNet-1k [20]. Il modello utilizza l'architettura ResNet-50 descritta nel Capitolo 3 ed è stato adattato alla classificazione binaria impostando il numero di classi di output a due, corrispondenti ai file benigni e malevoli. L'addestramento è stato eseguito per 4 epoche utilizzando l'ottimizzatore Adam, con un learning rate pari a 0.001 e un batch size di 32 immagini, mentre come funzione di loss è stata utilizzata la Cross-Entropy Loss, calcolata sia durante il training sia sul validation set al termine di ogni epoca. Se la validation loss risultava inferiore al valore migliore registrato nelle epoche precedenti, il modello veniva salvato sostituendo quello precedente, in modo da mantenere al termine dell'addestramento il modello con la validation loss più bassa. Durante l'addestramento, in alcune configurazioni è stato inoltre osservato che nelle ultime epoche la training loss continuava a diminuire mentre la validation loss iniziava ad aumentare. Ad esempio, nella configurazione SERPENTINE in scala di grigi a dimensione fissa, la training loss è passata da 0.0834 nella terza epoca a 0.0637 nella quarta, mentre la validation loss è aumentata leggermente da 0.1076 a 0.1082. Questo comportamento può indicare un iniziale fenomeno di overfitting e, poiché la validation loss della quarta epoca risultava superiore a quella della terza, il modello salvato al termine della terza epoca non è stato sostituito. Gli esperimenti sono stati eseguiti nell'ambiente Kaggle utilizzando le GPU messe a disposizione dalla piattaforma per l'addestramento e il test del modello.

4.2.3 Valutazione e metriche

Il modello salvato al termine dell'addestramento è stato quindi utilizzato sul test set per ottenere le predizioni finali e valutarne le prestazioni. Confrontando le etichette predette dal modello con quelle reali è stata costruita la matrice di confusione, da cui sono stati ricavati i valori dei veri positivi (*true positive*, TP), dei veri negativi (*true negative*, TN), dei falsi positivi (*false positive*, FP) e dei falsi negativi (*false negative*, FN). Considerando i file malevoli come classe positiva e quelli benigni come classe negativa, i veri positivi (TP) corrispondono ai file malevoli classificati correttamente e i veri negativi (TN) a quelli benigni classificati correttamente. I falsi positivi (FP) indicano invece i file benigni classificati come malevoli, mentre i falsi negativi (FN) indicano quelli malevoli classificati come benigni. A partire dai valori della matrice di confusione sono state calcolate precision, recall e F1-score per entrambe le classi. Per la classe malware, la precision indica la proporzione di file effettivamente malevoli tra quelli classificati come tali dal modello ed è calcolata come:

$$\text{Precision}_{malware} = \frac{TP}{TP + FP}$$

La recall indica invece la proporzione di file malevoli correttamente classificati rispetto al totale dei file malevoli:

$$\text{Recall}_{malware} = \frac{TP}{TP + FN}$$

Per la classe goodware, corrispondente ai file benigni, la precision indica la proporzione di file effettivamente benigni tra quelli classificati come tali dal modello ed è calcolata come:

$$\text{Precision}_{goodware} = \frac{TN}{TN + FN}$$

La recall indica invece la proporzione di file benigni correttamente classificati rispetto al totale dei file benigni:

$$\text{Recall}_{goodware} = \frac{TN}{TN + FP}$$

L'F1-score combina precision e recall attraverso la loro media armonica ed è calcolato, per entrambe le classi, come:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Infine, l'overall accuracy indica la proporzione di file classificati correttamente rispetto al numero totale di file analizzati ed è calcolata come:

$$\text{Overall Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Una volta completati i test, i valori delle matrici di confusione e le metriche calcolate a partire da essi sono stati salvati in un file CSV insieme ai principali parametri delle configurazioni sperimentali utilizzate, in modo da distinguere le diverse configurazioni e i corrispondenti risultati, per poi confrontarne le prestazioni.

4.3 Risultati

In questa sezione vengono presentati e confrontati i risultati ottenuti dalle 12 configurazioni sperimentali descritte nella Sezione 4.2. Le prestazioni sono state valutate sul test set attraverso la matrice di confusione e le metriche di precision, recall, F1-score e overall accuracy descritte in precedenza.

4.3.1 Risultati delle configurazioni sperimentali

Ciascuna delle 12 configurazioni sperimentali è stata valutata sullo stesso test set, composto da 40.310 file PE. Nelle seguenti Tabelle 4.3 e 4.4 sono riportati i valori della matrice di confusione e l'overall accuracy ottenuti rispettivamente dalle configurazioni a dimensione fissa 224×224 e ad altezza variabile.

Tabella 4.3: Risultati delle configurazioni a dimensione fissa

Mapping	**Rappr.**	**TN**	**FP**	**FN**	**TP**	**Overall accuracy**
LINEAR	RGB	16652	710	739	22209	0.9641
ZIGZAG	RGB	16783	579	958	21990	0.9619
SERPENTINE	RGB	16767	595	821	22127	0.9649
LINEAR	GRAY	16777	585	638	22310	0.9697
ZIGZAG	GRAY	16857	505	729	22219	0.9694
SERPENTINE	GRAY	16973	389	869	22079	0.9688

Tabella 4.4: Risultati delle configurazioni ad altezza variabile

Mapping	**Rappr.**	**TN**	**FP**	**FN**	**TP**	**Overall accuracy**
LINEAR	RGB	16951	411	584	22364	0.9753
ZIGZAG	RGB	17040	322	688	22260	0.9749
SERPENTINE	RGB	16826	536	427	22521	0.9761
LINEAR	GRAY	16888	474	565	22383	0.9742
ZIGZAG	GRAY	16979	383	699	22249	0.9732
SERPENTINE	GRAY	16863	499	512	22436	0.9749

Le Tabelle 4.5 e 4.6 riportano i valori di precision, recall e F1-score ottenuti per le classi goodware e malware, rispettivamente per le configurazioni a dimensione fissa 224×224 e ad altezza variabile.

Tabella 4.5: Metriche delle configurazioni a dimensione fissa

		Goodware			**Malware**		
Mapping	**Rappr.**	**Precision**	**Recall**	**F1-score**	**Precision**	**Recall**	**F1-score**
LINEAR	RGB	0.9575	0.9591	0.9583	0.9690	0.9678	0.9684
ZIGZAG	RGB	0.9460	0.9667	0.9562	0.9743	0.9583	0.9662
SERPENTINE	RGB	0.9533	0.9657	0.9595	0.9738	0.9642	0.9690
LINEAR	GRAY	0.9634	0.9663	0.9648	0.9744	0.9722	0.9733
ZIGZAG	GRAY	0.9585	0.9709	0.9647	0.9778	0.9682	0.9730
SERPENTINE	GRAY	0.9513	0.9776	0.9643	0.9827	0.9621	0.9723

Tabella 4.6: Metriche delle configurazioni ad altezza variabile

		Goodware			**Malware**		
Mapping	**Rappr.**	**Precision**	**Recall**	**F1-score**	**Precision**	**Recall**	**F1-score**
LINEAR	RGB	0.9667	0.9763	0.9715	0.9820	0.9746	0.9782
ZIGZAG	RGB	0.9612	0.9815	0.9712	0.9857	0.9700	0.9778
SERPENTINE	RGB	0.9753	0.9691	0.9722	0.9768	0.9814	0.9791
LINEAR	GRAY	0.9676	0.9727	0.9702	0.9793	0.9754	0.9773
ZIGZAG	GRAY	0.9605	0.9779	0.9691	0.9831	0.9695	0.9763
SERPENTINE	GRAY	0.9705	0.9713	0.9709	0.9782	0.9777	0.9780

Nel complesso, tutte le configurazioni hanno ottenuto valori elevati per le metriche considerate, con un'overall accuracy superiore a 0.96 in tutti gli esperimenti. Il valore più alto è stato ottenuto dalla configurazione SERPENTINE RGB ad altezza variabile, con un'overall accuracy pari a 0.9761.

4.3.2 Confronto delle configurazioni

Per confrontare le prestazioni ottenute dalle diverse configurazioni, nella seguente Tabella 4.7 sono riportati i valori di overall accuracy raggruppati in base al mapping, alla rappresentazione e alle dimensioni delle immagini.

Tabella 4.7: Confronto dell'overall accuracy delle configurazioni sperimentali

	Dimensione fissa		**Altezza variabile**	
Mapping	**RGB**	**GRAY**	**RGB**	**GRAY**
LINEAR	0.9641	0.9697	0.9753	0.9742
ZIGZAG	0.9619	0.9694	0.9749	0.9732
SERPENTINE	0.9649	0.9688	0.9761	0.9749

Dal confronto emerge innanzitutto che le configurazioni ad altezza variabile hanno ottenuto un'overall accuracy superiore alle corrispondenti configurazioni a dimensione fissa per tutti e tre i mapping, considerando sia la rappresentazione RGB sia quella in scala di grigi. Nel caso della rappresentazione RGB, l'overall accuracy è aumentata di 1,12 punti percentuali per LINEAR, 1,30 punti percentuali per ZIGZAG e 1,12 punti percentuali per SERPENTINE dalla configurazione a dimensione fissa a quella ad altezza variabile. Per la rappresentazione in scala di grigi, l'aumento è stato invece pari a 0,45 punti percentuali per LINEAR, 0,38 per ZIGZAG e 0,61 per SERPENTINE. Confrontando le due rappresentazioni, nelle configurazioni a dimensione fissa la scala di grigi ha ottenuto un'overall accuracy superiore rispetto alla rappresentazione RGB per tutti e tre i mapping, in particolare con una differenza di 0,56 punti percentuali per LINEAR, 0,75 per ZIGZAG e 0,39 per SERPENTINE. Nelle configurazioni ad altezza variabile si osserva invece un andamento opposto, con la rappresentazione RGB che ha ottenuto un'overall

accuracy superiore rispetto alla scala di grigi per tutti e tre i mapping, in particolare con una differenza di 0,11 punti percentuali per LINEAR, 0,17 per ZIGZAG e 0,12 per SERPENTINE.

Per quanto riguarda le strategie di mapping, le differenze risultano più contenute e non emerge una strategia che ottiene risultati nettamente migliori rispetto alle altre. Tra le configurazioni a dimensione fissa, LINEAR ha ottenuto il valore più elevato nella rappresentazione in scala di grigi, con un'overall accuracy pari a 0.9697, mentre SERPENTINE ha ottenuto il risultato migliore nella rappresentazione RGB, con un valore pari a 0.9649. In particolare, tra i tre mapping si registra una differenza massima di 0,09 punti percentuali per la rappresentazione in scala di grigi e di 0,30 punti percentuali per quella RGB, considerando in ciascun caso il valore dell'overall accuracy più elevato e quello più basso. Anche nelle configurazioni ad altezza variabile le differenze tra i mapping sono limitate. SERPENTINE ha ottenuto il valore più elevato sia con la rappresentazione RGB sia con quella in scala di grigi, con un'overall accuracy pari rispettivamente a 0.9761 e 0.9749. In particolare, tra i tre mapping si registra una differenza massima di 0,12 punti percentuali per la rappresentazione RGB e di 0,17 punti percentuali per quella in scala di grigi.

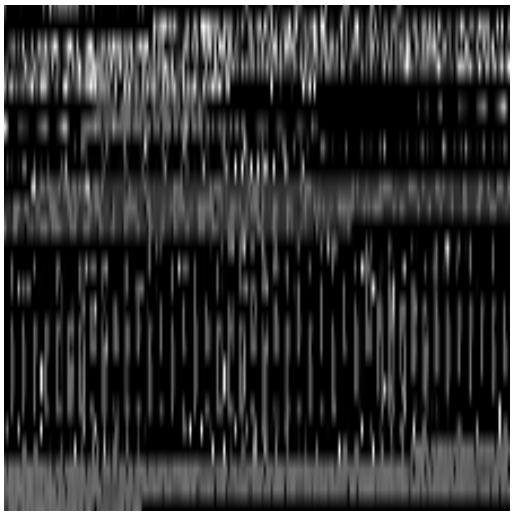
Considerando complessivamente le 12 configurazioni sperimentali, il valore più elevato di overall accuracy è stato quindi ottenuto dalla configurazione SERPENTINE RGB ad altezza variabile, con un valore pari a 0.9761, corrispondente al 97,61%.

4.3.3 Discussione dei risultati

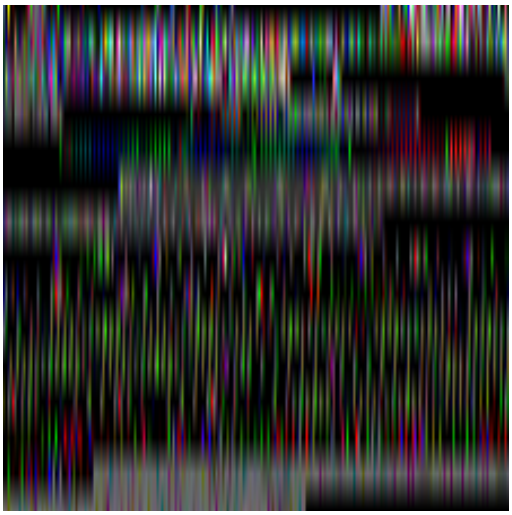
I risultati raccolti mostrano che le configurazioni ad altezza variabile hanno ottenuto prestazioni superiori rispetto a quelle a dimensione fissa per tutti e tre i mapping e per entrambe le rappresentazioni. Questo risultato può essere ricondotto alle differenze nella rappresentazione dei file PE nelle due configurazioni, infatti nelle configurazioni a dimensione fissa il numero di pixel disponibili è prestabilito e limita quindi la quantità di byte che può essere rappresentata, mentre nelle configurazioni ad altezza variabile l'altezza viene determinata in base alla dimensione del singolo file, permettendo di rappresentarne tutti i byte prima del successivo resize a 224×224 pixel. Inoltre, nel caso dei file PE di dimensioni ridotte, nelle

configurazioni a dimensione fissa la parte dell'immagine non occupata dai byte del file viene completata mediante zero-padding, come mostrato precedentemente nella Figura 4.1, mentre nelle configurazioni ad altezza variabile l'altezza si adatta alla dimensione del file, rendendo quindi non necessario l'utilizzo dello zero-padding, che potrebbe influire sulle prestazioni del modello.

Confrontando le rappresentazioni RGB e in scala di grigi, nelle configurazioni a dimensione fissa la rappresentazione in scala di grigi ha ottenuto prestazioni migliori per tutti e tre i mapping, mentre nelle configurazioni ad altezza variabile si osserva il comportamento opposto, con risultati leggermente migliori per la rappresentazione RGB. Una possibile spiegazione di questo comportamento può essere legata al diverso numero di byte rappresentati da ciascun pixel, in particolare nella rappresentazione RGB ogni pixel contiene tre byte del file PE, mentre nella rappresentazione in scala di grigi ne contiene uno. Nelle configurazioni a dimensione fissa, entrambe pari a 224×224 pixel, questo comporta che la parte iniziale del file occupi una porzione maggiore dell'immagine nella rappresentazione in scala di grigi, aspetto che potrebbe aver contribuito ai risultati ottenuti. Nelle configurazioni ad altezza variabile, invece, le due rappresentazioni richiedono un diverso numero totale di pixel per rappresentare lo stesso file e, insieme alle diverse larghezze utilizzate, pari a 457 pixel per RGB e 792 pixel per la scala di grigi, producono immagini con dimensioni diverse prima del resize a 224×224 pixel. Di conseguenza, durante il ridimensionamento può verificarsi una deformazione differente delle immagini, che potrebbe influire sulle prestazioni ottenute. Un esempio delle due rappresentazioni ottenute dallo stesso file PE è mostrato nella Figura 4.2.



(a) Rappresentazione GRAY



(b) Rappresentazione RGB

Figura 4.2: Immagini dello stesso file PE nelle rappresentazioni GRAY e RGB con mapping LINEAR e altezza variabile, dopo il resize a 224×224 pixel

Per quanto riguarda le strategie di mapping, nonostante le differenze contenute osservate tra le tre strategie, SERPENTINE ha ottenuto le prestazioni migliori in tre delle quattro combinazioni considerate tra rappresentazione e dimensioni delle immagini. Questo risultato potrebbe essere legato alla disposizione dei byte utilizzata da SERPENTINE che, come descritto nel Capitolo 3, permette di concentrare in una stessa area dell'immagine le informazioni appartenenti alla parte iniziale del file PE, invece di distribuirle lungo le prime righe dell'immagine. Questa diversa disposizione delle informazioni potrebbe aver contribuito alle prestazioni di SERPENTINE anche se le differenze tra i mapping rimangono limitate.

Nel complesso, i risultati indicano che la configurazione delle dimensioni delle immagini ha avuto un'influenza maggiore sulle prestazioni rispetto alla strategia di mapping utilizzata, per la quale sono state osservate differenze più contenute.

Capitolo 5

Conclusioni e sviluppi futuri

In questo lavoro di tesi è stato affrontato il problema della classificazione di file Windows PE come benigni o malevoli attraverso la loro rappresentazione sotto forma di immagini e tramite l'utilizzo di una rete ResNet-50. In particolare, sono state confrontate le strategie di mapping LINEAR, ZIGZAG e SERPENTINE, considerando sia immagini RGB sia immagini in scala di grigi e le due diverse strategie relative alle dimensioni, a dimensione fissa e ad altezza variabile. Nel complesso, i risultati ottenuti hanno mostrato prestazioni elevate per tutte le configurazioni considerate, evidenziando risultati migliori per le configurazioni ad altezza variabile rispetto alle corrispondenti configurazioni a dimensione fissa. In particolare, la migliore overall accuracy è stata ottenuta dalla configurazione SERPENTINE RGB ad altezza variabile, con un valore pari al 97,61%. L'approccio basato sulla rappresentazione dei file PE come immagini e sulla loro classificazione mediante ResNet-50 ha permesso di ottenere buone prestazioni sul dataset considerato, mostrando inoltre come la scelta delle dimensioni delle immagini possa avere un ruolo importante nelle prestazioni del modello.

Come possibile sviluppo futuro, l'analisi potrebbe essere estesa utilizzando dataset differenti, in modo da verificare se le prestazioni rimangano coerenti anche su altri insiemi di file PE. Un ulteriore sviluppo potrebbe riguardare il confronto dell'accuratezza della classificazione ottenuta con la rete ResNet-50 con l'accuratezza che si potrebbe ottenere usando altre architetture della stessa famiglia, caratterizzate da una diversa profondità, in modo da valutare l'influenza del numero di livelli sulle prestazioni ottenute. Potrebbero inoltre essere valutate diverse modalità per

determinare le dimensioni delle immagini, ad esempio calcolandole in funzione della lunghezza in byte del singolo file PE, invece di utilizzare una larghezza comune determinata a partire dalla lunghezza media in byte dei file del dataset. Nella versione attuale della tesi, la fase di pre-elaborazione delle immagini, compresa la fase di ridimensionamento (resize) delle immagini, è stata eseguita prima di fornire le immagini in input alla ResNet-50, mentre potrebbe essere interessante valutare come cambiano le prestazioni utilizzando l'AutoImageProcessor di Hugging Face [21] per la preparazione delle immagini. Questi possibili sviluppi permetterebbero di proseguire l'analisi e approfondire, in futuro, come la rappresentazione dei file PE come immagini possa contribuire a migliorare l'accuratezza della classificazione di file benigni e malevoli.

Appendice A

Progettazione UML

A.1 Diagramma UML generale

Il diagramma riportato nella Figura A.1 mostra la struttura complessiva del progetto, evidenziando le classi, i package di appartenenza e le relazioni tra le classi.

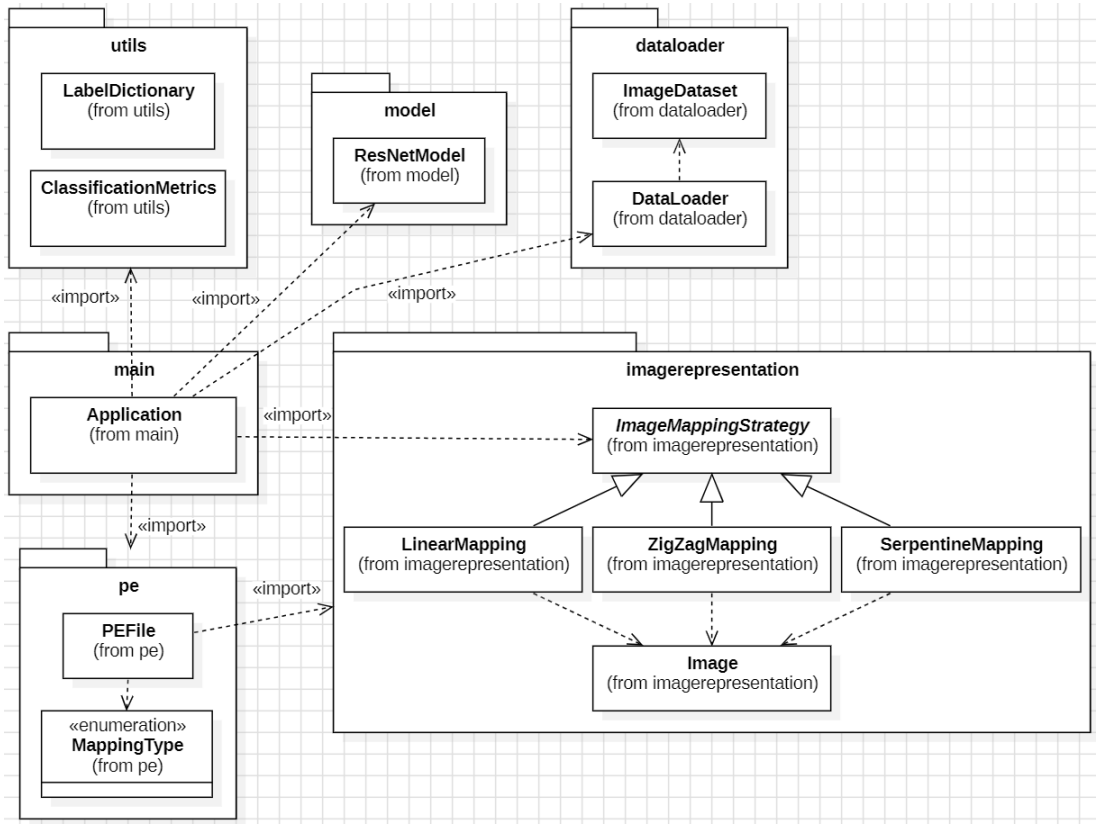


Figura A.1: Diagramma UML generale del progetto

A.2 Package main

Il package `main` (Figura A.2) contiene il punto di ingresso del programma e coordina l'esecuzione della pipeline. Al suo interno vengono letti i parametri del file di configurazione e vengono avviate le principali fasi del processo.

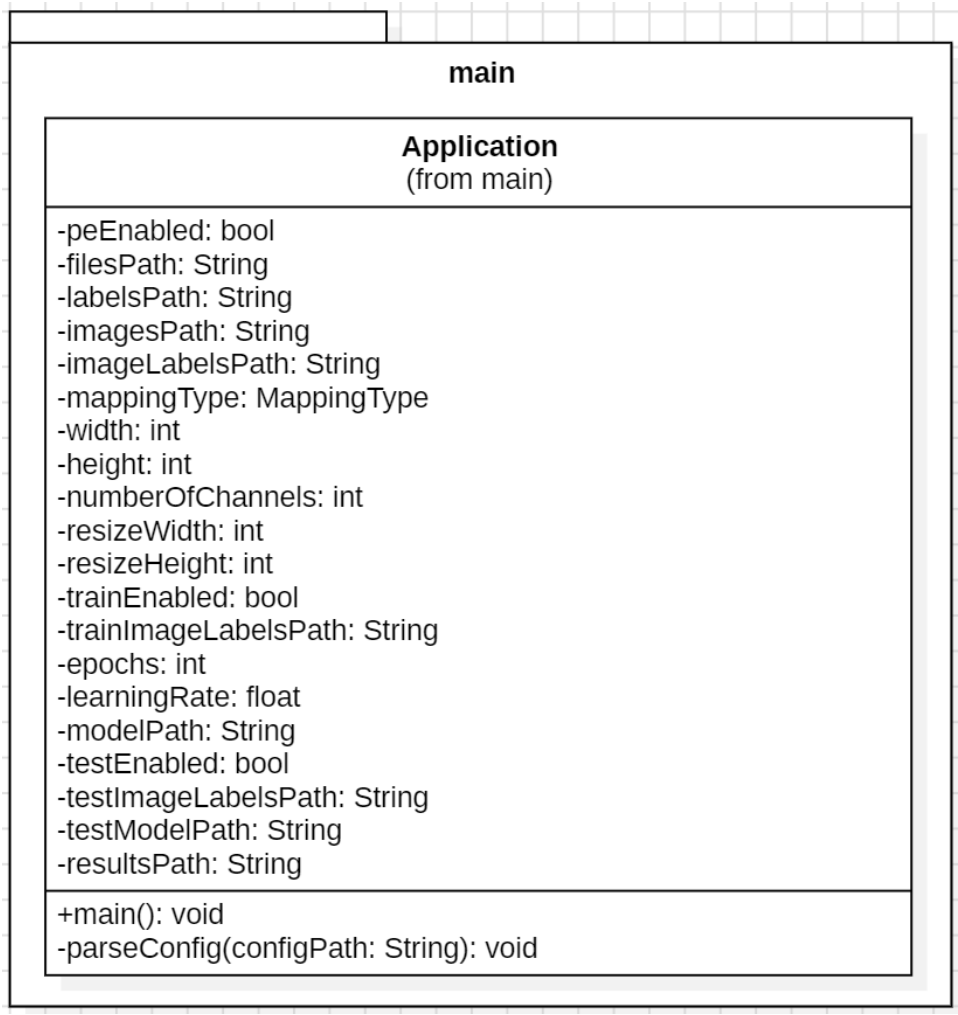


Figura A.2: Diagramma UML del package `main`

A.2.1 Classe Application

La classe `Application`, appartenente al package `main`, è la classe principale del programma e si occupa di leggere il file di configurazione e, in base ai parametri impostati, di coordinarne le diverse fasi. Per svolgere le varie operazioni, richiama le classi responsabili della conversione dei file PE, della creazione del file CSV con

le etichette, del training e del test del modello. Durante la fase di test, coordina anche il calcolo delle metriche e il salvataggio dei risultati.

Attributi

- **peEnabled** (`bool`): indica se abilitare la fase di conversione PE in immagini.
- **filesPath** (`String`): percorso della cartella contenente i file PE da convertire.
- **labelsPath** (`String`): percorso del file CSV contenente i nomi dei file PE e le relative etichette.
- **imagesPath** (`String`): percorso della cartella in cui vengono salvate le immagini generate.
- **imageLabelsPath** (`String`): percorso del file CSV in cui viene salvato il mapping tra i percorsi delle immagini generate e le relative etichette.
- **mappingType** (`MappingType`): strategia di mapping da utilizzare.
- **width** (`int`): larghezza delle immagini da generare.
- **height** (`int`): altezza delle immagini da generare.
- **numberOfChannels** (`int`): numero di canali delle immagini da generare.
- **resizeWidth** (`int`): larghezza per l'eventuale resize.
- **resizeHeight** (`int`): altezza per l'eventuale resize.
- **trainEnabled** (`bool`): indica se abilitare la fase di training del modello.
- **trainImageLabelsPath** (`String`): percorso del file CSV contenente il mapping immagini-etichette utilizzato durante il training.
- **epochs** (`int`): numero di epoche del training.
- **learningRate** (`float`): learning rate utilizzato durante il training.

- **modelPath** (`String`): percorso della cartella in cui viene salvato il modello addestrato.
- **testEnabled** (`bool`): indica se abilitare la fase di test del modello addestrato.
- **testImageLabelsPath** (`String`): percorso del file CSV contenente il mapping immagini-etichette utilizzato durante il test.
- **testModelPath** (`String`): percorso del modello addestrato da utilizzare durante il test.
- **resultsPath** (`String`): percorso del file CSV in cui vengono salvati i risultati del test e le metriche calcolate.

Metodi

- **main()** → `void`: punto di ingresso del programma. Coordina l'esecuzione delle diverse fasi in base ai parametri caricati dal file di configurazione.
- **parseConfig**(`configPath:String`) → `void`: metodo privato che legge il file di configurazione e inizializza gli attributi necessari per l'esecuzione delle diverse fasi del programma.

A.3 Package pe

Il package `pe` (Figura A.3) si occupa della gestione dei file Portable Executable di Windows. Il suo compito principale è leggere i file PE in formato binario e avviare il processo di conversione in immagini, utilizzando la strategia di mapping ricevuta dalla classe `Application`.

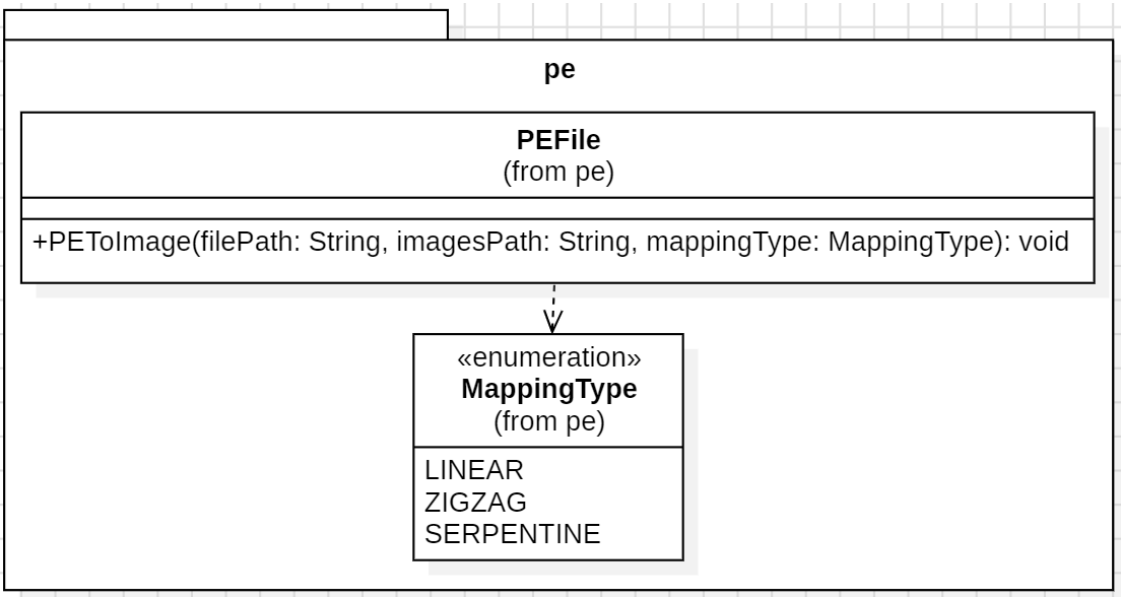


Figura A.3: Diagramma UML del package `pe`

A.3.1 Classe PEFile

La classe `PEFile`, appartenente al package `pe`, è responsabile della conversione di un singolo file PE. Riceve il percorso del file, il percorso della cartella in cui salvare l'immagine e il valore di `MappingType`. Dopo aver letto il contenuto binario del file, utilizza il valore di `MappingType` ricevuto per individuare la strategia di mapping da applicare e salva infine l'immagine ottenuta nella cartella specificata.

Metodi

- **PEToImage**(`filePath:String`, `imagesPath:String`, `mappingType:MappingType`) → `void`: legge il singolo file PE indicato, passa la sequenza di byte al metodo `createImage` della strategia di mapping corrispondente e salva l'immagine ottenuta nella cartella specificata.

A.3.2 Enumerazione MappingType

L'enumerazione `MappingType`, appartenente al package `pe`, definisce il tipo di strategia di mapping da utilizzare per la conversione dei file PE in immagini. La classe `Application` legge il valore dal file di configurazione e lo passa alla classe `PEFile`,

che lo utilizza per determinare la strategia di mapping da applicare durante la generazione delle immagini.

Valori

- **LINEAR**
- **ZIGZAG**
- **SERPENTINE**

A.4 Package image_representation

Il package `image_representation` contiene le funzionalità necessarie per rappresentare i file PE sotto forma di immagini. Al suo interno sono definite la classe `Image`, che rappresenta l'immagine e la sua struttura, e le diverse strategie di mapping utilizzate per disporre i byte del file nell'immagine.

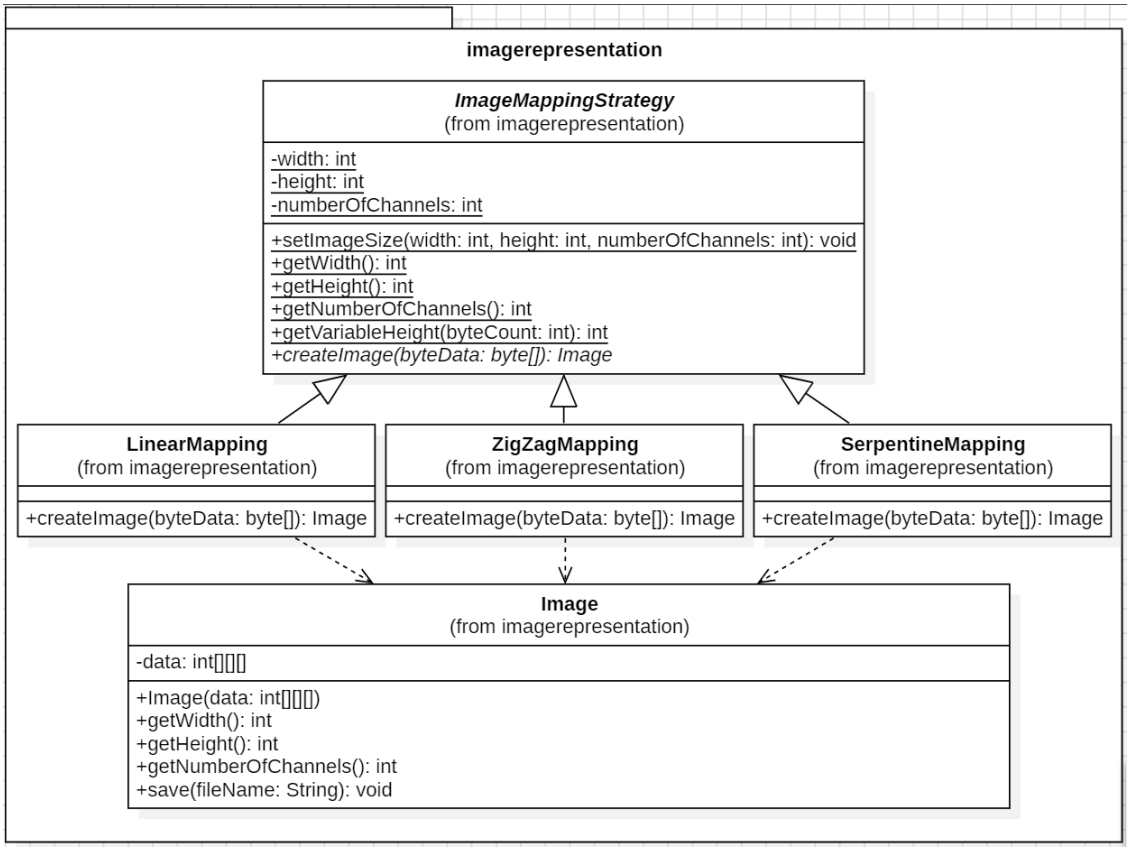


Figura A.4: Diagramma UML del package `image_representation`

A.4.1 Classe Image

La classe `Image`, appartenente al package `image_representation`, rappresenta l'immagine generata dalla conversione del file PE. L'immagine è rappresentata tramite un tensore a tre dimensioni che contiene i valori dei pixel organizzati secondo righe, colonne e canali. La classe riceve il tensore già costruito dalla strategia di mapping e si occupa della sua gestione e del salvataggio su file.

Attributi

- **data** (`int[][][]`): tensore tridimensionale che contiene i valori dell'immagine organizzati in altezza, larghezza e numero di canali.

Metodi

- **Image**(`data:int[][][]`): costruttore della classe che inizializza l'immagine.

- **getWidth**() → `int`: restituisce la larghezza dell'immagine ricavandola dal tensore associato.
- **getHeight**() → `int`: restituisce l'altezza dell'immagine ricavandola dal tensore associato.
- **getNumberOfChannels**() → `int`: restituisce il numero di canali dell'immagine ricavandolo dal tensore associato.
- **save**(`fileName:String`) → `void`: salva l'immagine rappresentata dal tensore associato nel percorso specificato. In base al numero di canali, l'immagine viene salvata in grayscale oppure in formato RGB.

A.4.2 Classe astratta ImageMappingStrategy

La classe astratta `ImageMappingStrategy`, appartenente al package `image_representation`, definisce il comportamento comune delle strategie di mapping. Contiene i parametri necessari alla costruzione dell'immagine, cioè larghezza, altezza e numero di canali, impostati prima della fase di creazione dell'immagine. Ogni implementazione riceve in input la sequenza di byte del file PE e costruisce l'immagine secondo la propria strategia di mapping. L'altezza può essere definita nel file di configurazione oppure impostata a 0 per utilizzare la modalità ad altezza variabile. In questo caso, l'altezza necessaria viene calcolata durante la creazione dell'immagine senza modificare la configurazione della strategia. La gestione dei canali permette di costruire immagini sia in grayscale (1 canale) sia RGB (3 canali), modificando il modo in cui i byte vengono letti e inseriti nel tensore, in particolare un byte per pixel nel caso grayscale oppure gruppi di tre byte per pixel nel caso RGB.

Attributi

- **width** (`int`): attributo di classe che rappresenta la larghezza comune utilizzata per la costruzione delle immagini.
- **height** (`int`): attributo di classe che rappresenta l'altezza comune utilizzata per la costruzione delle immagini.

- **numberOfChannels** (`int`): attributo di classe che rappresenta il numero di canali utilizzato per definire la struttura del tensore.

Metodi

- **setImageSize**(`width:int`, `height:int`, `numberOfChannels:int`) → `void`: metodo di classe che imposta i parametri comuni utilizzati dalle strategie per costruire l'immagine.
- **getVariableHeight**(`byteCount:int`) → `int`: metodo di classe che calcola e restituisce l'altezza necessaria in base al numero di byte da rappresentare, quando l'altezza non è definita.
- **getWidth**() → `int`: metodo di classe che restituisce la larghezza configurata.
- **getHeight**() → `int`: metodo di classe che restituisce l'altezza configurata.
- **getNumberOfChannels**() → `int`: metodo di classe che restituisce il numero di canali configurato.
- **createImage**(`byteData:byte[]`) → `Image`: metodo astratto che viene implementato dalle classi derivate in base alla strategia di mapping utilizzata e che costruisce e restituisce un oggetto della classe `Image` a partire dalla sequenza di byte del file PE.

A.4.3 Classe LinearMapping

La classe `LinearMapping`, appartenente al package `image_representation`, eredita dalla classe astratta `ImageMappingStrategy` e implementa il mapping disponendo i byte nel tensore in modo sequenziale, da sinistra verso destra e dall'alto verso il basso.

Metodi

- **createImage**(`byteData:byte[]`) → `Image`: implementa il metodo astratto della classe `ImageMappingStrategy` e costruisce l'immagine utilizzando il mapping Linear.

A.4.4 Classe ZigZagMapping

La classe `ZigZagMapping`, appartenente al package `image_representation`, eredita dalla classe astratta `ImageMappingStrategy` e implementa il mapping disponendo i byte nel tensore in modo sequenziale e alternando il verso di percorrenza delle righe.

Metodi

- **createImage**(`byteData:byte[]`) → `Image`: implementa il metodo astratto della classe `ImageMappingStrategy` e costruisce l'immagine utilizzando il mapping Zigzag.

A.4.5 Classe SerpentineMapping

La classe `SerpentineMapping`, appartenente al package `image_representation`, eredita dalla classe astratta `ImageMappingStrategy` e implementa il mapping disponendo i byte nel tensore lungo le diagonali e alternando la direzione di percorrenza.

Metodi

- **createImage**(`byteData:byte[]`) → `Image`: implementa il metodo astratto della classe `ImageMappingStrategy` e costruisce l'immagine utilizzando il mapping Serpentine.

A.5 Package dataloader

Il package `dataloader` si occupa di preparare le immagini e le relative etichette per essere utilizzate dal modello. Gestisce la creazione dei dataset necessari per il training, la validation e il test.

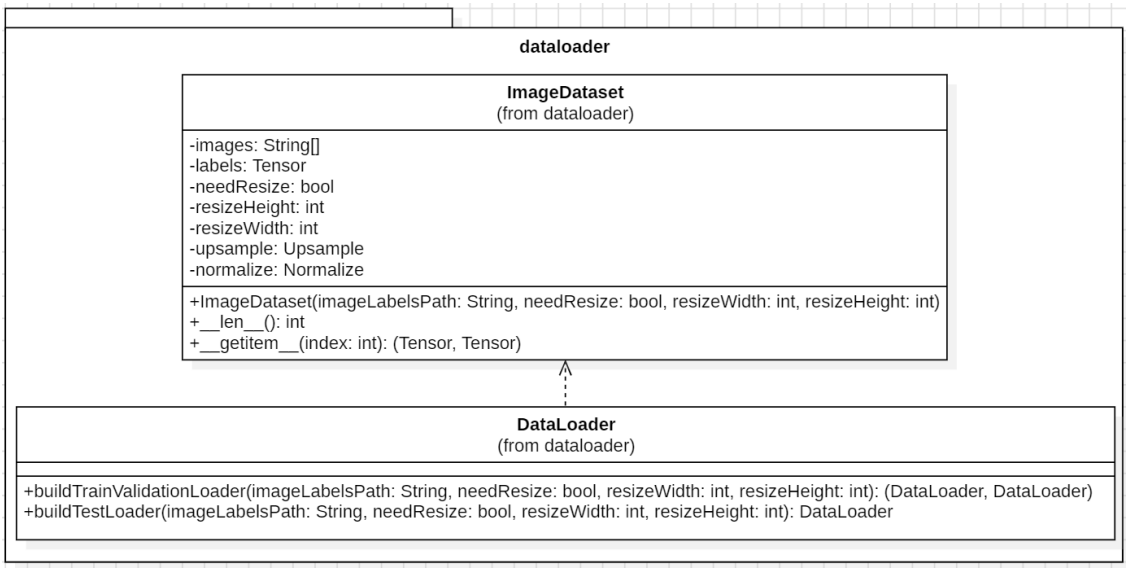


Figura A.5: Diagramma UML del package `dataloader`

A.5.1 Classe ImageDataset

La classe `ImageDataset`, appartenente al package `dataloader`, rappresenta il dataset, eredita da `torch.utils.data.Dataset` e si occupa della gestione delle immagini generate dalla conversione dei file PE e delle relative etichette contenute nel file CSV. Il suo compito è fornire le coppie immagine-etichetta richieste dal `DataLoader` e, se necessario, applicare il resize delle immagini e la normalizzazione prevista per il modello ResNet.

Attributi

- **images** (`String[]`): array contenente i percorsi completi delle immagini letti dal file CSV.
- **labels** (`Tensor`): tensore PyTorch contenente le etichette associate alle immagini.
- **needResize** (`bool`): indica se deve essere applicato il resize alle immagini.
- **resizeWidth** (`int`): larghezza per l'eventuale resize.
- **resizeHeight** (`int`): altezza per l'eventuale resize.

- **upsample** (`nn.Upsample`): modulo utilizzato per ridimensionare le immagini quando il resize è abilitato.
- **normalize** (`Normalize`): trasformazione utilizzata per normalizzare i valori dei pixel prima di passarli al modello.

Metodi

- **ImageDataset**(`imageLabelsPath:String`, `needResize:bool`, `resizeWidth:int`, `resizeHeight:int`): costruttore della classe che legge il file CSV, estrae i percorsi delle immagini e le etichette e inizializza i parametri necessari per il caricamento, la normalizzazione e l'eventuale resize delle immagini.
- **len**() → `int`: restituisce il numero totale di elementi presenti nel dataset.
- **getitem**(`index:int`) → `(Tensor, Tensor)`: recupera l'immagine e l'etichetta corrispondente all'indice indicato. L'immagine viene convertita in un tensore, normalizzata ed eventualmente ridimensionata.

A.5.2 Classe DataLoader

La classe `DataLoader`, appartenente al package `dataloader`, prepara i dati che vengono utilizzati dal modello durante le fasi di training, validation e test. A partire dal percorso del file CSV e dai parametri relativi al resize, crea un oggetto della classe `ImageDataset` e lo utilizza per costruire i `DataLoader` di PyTorch. Durante il training, il dataset viene diviso in due parti, una utilizzata per il training e una per la validation. Durante il test, invece, viene utilizzato un unico dataset contenente le immagini da classificare.

Metodi

- **buildTrainValidationLoader** (`imageLabelsPath:String`, `needResize:bool`, `resizeWidth:int`, `resizeHeight:int`) → `(torch.utils.data.DataLoader, torch.utils.data.DataLoader)`: crea il dataset, lo divide in training e validation e restituisce due oggetti `torch.utils.data.DataLoader`: `trainLoader` e `validationLoader`.

- **buildTestLoader**(`imageLabelsPath:String`, `needResize:bool`, `resizeWidth:int`, `resizeHeight:int`) → `torch.utils.data.DataLoader`: crea il dataset di test e restituisce un oggetto `torch.utils.data.DataLoader`, denominato `testLoader`.

A.6 Package model

Il package `model` contiene le funzionalità relative al modello di classificazione. Si occupa dell'addestramento, della validazione e del test della rete neurale utilizzata per distinguere i file goodware dai file malware.

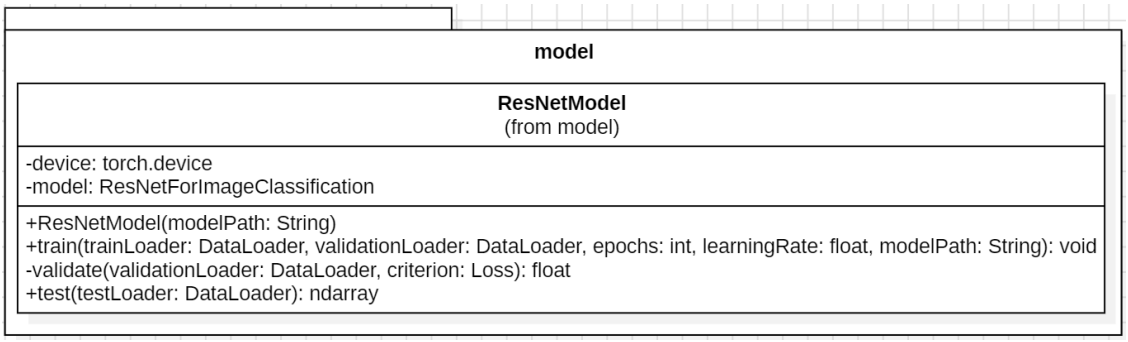


Figura A.6: Diagramma UML del package `model`

A.6.1 Classe ResNetModel

La classe `ResNetModel`, appartenente al package `model`, è responsabile della gestione del modello di classificazione basato su una ResNet-50 preaddestrata. Si occupa dell'inizializzazione del modello, dell'addestramento, della validazione e del test. Durante il training salva il modello che ottiene la validation loss più bassa, mentre durante il test restituisce la matrice di confusione ottenuta confrontando le etichette reali con quelle predette.

Attributi

- **device** (`torch.device`): dispositivo utilizzato per eseguire il modello (CPU o GPU).
- **model** (`ResNetForImageClassification`): modello ResNet-50 utilizzato per la classificazione delle immagini.

Metodi

- **ResNetModel**(`modelPath:String`): costruttore della classe. Se il percorso del modello non viene specificato, inizializza una ResNet-50 preaddestrata, altrimenti carica un modello precedentemente salvato.
- **train**(`trainLoader:DataLoader`, `validationLoader:DataLoader`, `epochs:int`, `learningRate:float`, `modelPath:String`) → `void`: esegue il training del modello, calcola la training loss e la validation loss a ogni epoca e salva automaticamente il modello che ottiene la validation loss più bassa.
- **validate**(`validationLoader:DataLoader`, `criterion:CrossEntropyLoss`) → `float`: metodo privato che esegue la fase di validazione e restituisce la validation loss media.
- **test**(`testLoader:DataLoader`) → `ndarray`: esegue il test del modello e restituisce la matrice di confusione ottenuta confrontando le etichette reali con quelle predette.

A.7 Package utils

Il package `utils` contiene alcune funzionalità di supporto al progetto. In particolare, gestisce la creazione del file CSV con il mapping tra i percorsi delle immagini e le relative etichette e il calcolo delle metriche a partire dalla matrice di confusione ottenuta durante la classificazione.

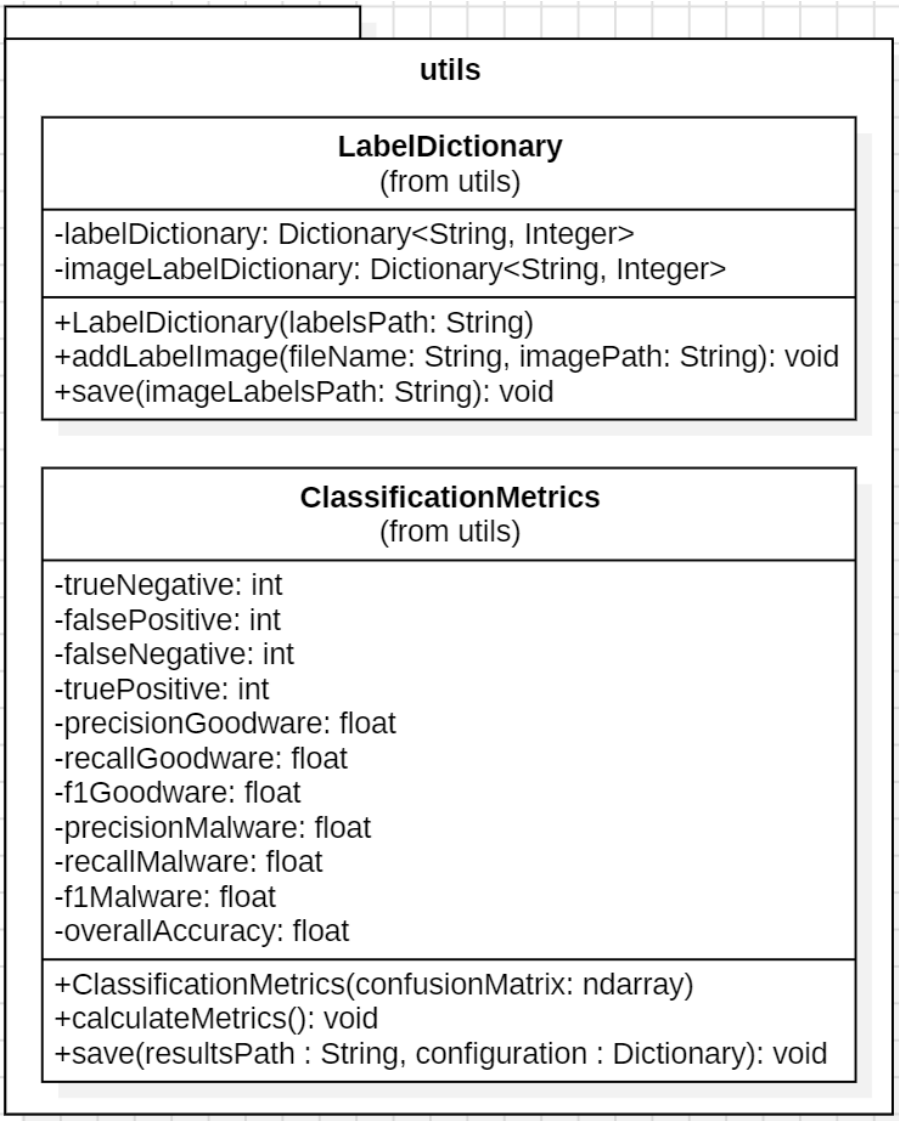


Figura A.7: Diagramma UML del package `utils`

A.7.1 Classe LabelDictionary

La classe `LabelDictionary`, appartenente al package `utils`, è responsabile della gestione delle etichette dei file PE e del mapping tra le immagini generate e le relative etichette. Contiene due dizionari privati: il primo associa il nome del file PE alla relativa etichetta, mentre il secondo associa il percorso dell'immagine generata alla relativa etichetta. Inizializza il primo dizionario nel costruttore, leggendo le etichette presenti nel file CSV indicato, e costruisce durante la conversione il mapping nel secondo dizionario, che successivamente viene salvato in un file CSV nel

percorso specificato.

Attributi

- **labelDictionary** (`Dictionary<String, Integer>`): dizionario privato che associa il nome del file PE alla relativa etichetta.
- **imageLabelDictionary** (`Dictionary<String, Integer>`): dizionario privato che associa il percorso dell'immagine alla relativa etichetta.

Metodi

- **LabelDictionary**(`labelsPath:String`): costruttore della classe. Legge il file CSV delle etichette e inizializza il primo dizionario.
- **addLabelImage**(`fileName:String`, `imagePath:String`) → `void`: recupera l'etichetta del file PE e aggiunge il mapping tra immagine e relativa etichetta al secondo dizionario.
- **save**(`imageLabelsPath:String`) → `void`: salva il secondo dizionario in un file CSV.

A.7.2 Classe ClassificationMetrics

La classe `ClassificationMetrics`, appartenente al package `utils`, è responsabile del calcolo delle metriche di classificazione e del loro salvataggio nel file CSV indicato. Riceve la matrice di confusione restituita dal test del modello ed estrae i quattro valori TN, FP, FN e TP. A partire da questi valori calcola l'overall accuracy e, per le classi goodware e malware, la precision, il recall e l'F1-score.

Attributi

- **trueNegative** (`int`): numero di campioni goodware classificati correttamente.
- **falsePositive** (`int`): numero di campioni goodware classificati come malware.

- **falseNegative** (`int`): numero di campioni malware classificati come goodware.
- **truePositive** (`int`): numero di campioni malware classificati correttamente.
- **precisionGoodware** (`float`): precision calcolata per la classe goodware.
- **recallGoodware** (`float`): recall calcolata per la classe goodware.
- **f1Goodware** (`float`): F1-score calcolato per la classe goodware.
- **precisionMalware** (`float`): precision calcolata per la classe malware.
- **recallMalware** (`float`): recall calcolata per la classe malware.
- **f1Malware** (`float`): F1-score calcolato per la classe malware.
- **overallAccuracy** (`float`): overall accuracy calcolata.

Metodi

- **ClassificationMetrics**(`confusionMatrix:ndarray`): costruttore della classe. Riceve la matrice di confusione ed estrae i valori TN, FP, FN e TP.
- **calculateMetrics**() → `void`: calcola l'overall accuracy e, per le classi goodware e malware, la precision, il recall e l'F1-score.
- **save**(`resultsPath:String`, `configuration:Dictionary`) → `void`: salva i valori della matrice di confusione e le metriche calcolate insieme alla configurazione del modello testato in una riga del file CSV dei risultati.

Bibliografia

[1] Microsoft. PE format. `https://learn.microsoft.com/en-us/windows/win32/debug/pe-format`.

[2] Faiza Babar Khan, Muhammad Hanif Durad, Asifullah Khan, Farrukh Aslam Khan, Sajjad Hussain Chauhdary, and Mohammed Alqarni. Detection of data scarce malware using one-shot learning with relation network. *IEEE Access*, 11:74438–74457, 2023.

[3] Yunchun Zhang, Jiaqi Jiang, Chao Yi, Hai Li, Shaohui Min, Ruifeng Zuo, Zhenzhou An, and Yongtao Yu. A robust CNN for malware classification against executable adversarial attack. *Electronics*, 13(5):989, 2024.

[4] Jinrong Bai, Junfeng Wang, and Guozhong Zou. A malware detection scheme based on mining format information. *The Scientific World Journal*, 2014(1):260905, 2014.

[5] Murugiah Souppaya and Karen Scarfone. Guide to malware incident prevention and handling for desktops and laptops. Technical Report NIST SP 800-83 Rev. 1, National Institute of Standards and Technology, 2013.

[6] Manuel Egele, Theodoor Scholte, Engin Kirda, and Christopher Kruegel. A survey on automated dynamic malware-analysis techniques and tools. *ACM Computing Surveys*, 44(2):1–42, 2012.

[7] Simon Mutithu. Paper 1: Static and dynamic malware analysis, April 2026. Comparative Analysis of Static and Dynamic Malware Analysis Techniques.

[8] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. Survey of machine learning techniques for malware analysis. *Computers & Security*, 81:123–147, 2019.

[9] Lakshmanan Nataraj, Shanmugavadivel Karthikeyan, Grégoire Jacob, and

B. S. Manjunath. Malware images: Visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, pages 1–7. Association for Computing Machinery, 2011.

[10] Hui Guo, Shuguang Huang, Cheng Huang, Fan Shi, Min Zhang, and Zulie Pan. Binary file's visualization and entropy features analysis combined with multiple deep learning networks for malware classification. *Security and Communication Networks*, 2020:8881760, 2020.

[11] Vo Khuong Linh, Nguyen Viet Hung, Tran Ngoc Anh, Duong Do Nhuan, and Dinh Cong Hien. Enhance deep learning model for malware detection with a new image representation method. *Journal of Science and Technology on Information Security*, 1(21):31–39, 2024.

[12] Huy Nguyen, Fabio Di Troia, Genya Ishigaki, and Mark Stamp. Generative adversarial networks and image-based malware classification. *Journal of Computer Virology and Hacking Techniques*, 19(4):579–595, 2023.

[13] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[14] Kh. Nafizul Haque. What is convolutional neural network — cnn (deep learning). Medium, February 2023. `https://nafizshahriar.medium.com/what-is-convolutional-neural-network-cnn-deep-learning-b3921bdd82d5`.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[16] Michael Lester. PE malware machine learning dataset, 2021. Practical Security Analytics. `https://practicalsecurityanalytics.com/pe-malware-machine-learning-dataset/`.

[17] PyTorch. PyTorch documentation. Documentazione ufficiale. `https://docs.pytorch.org/`.

[18] PyTorch. Torchvision. `https://pytorch.org/vision/stable/index.html`.

[19] Hugging Face. Preprocessor configuration for microsoft/resnet-50. Configurazione del preprocessing del modello microsoft/resnet-50. `https://huggingface.co/microsoft/resnet-50/blob/main/preprocessor_config.json`.

[20] Hugging Face. ResNet-50 v1.5. Model card del modello microsoft/resnet-50. `https://huggingface.co/microsoft/resnet-50`.

[21] Hugging Face. Image processors. Documentazione degli Image Processor di Hugging Face. `https://huggingface.co/docs/transformers/main/image_processors`.